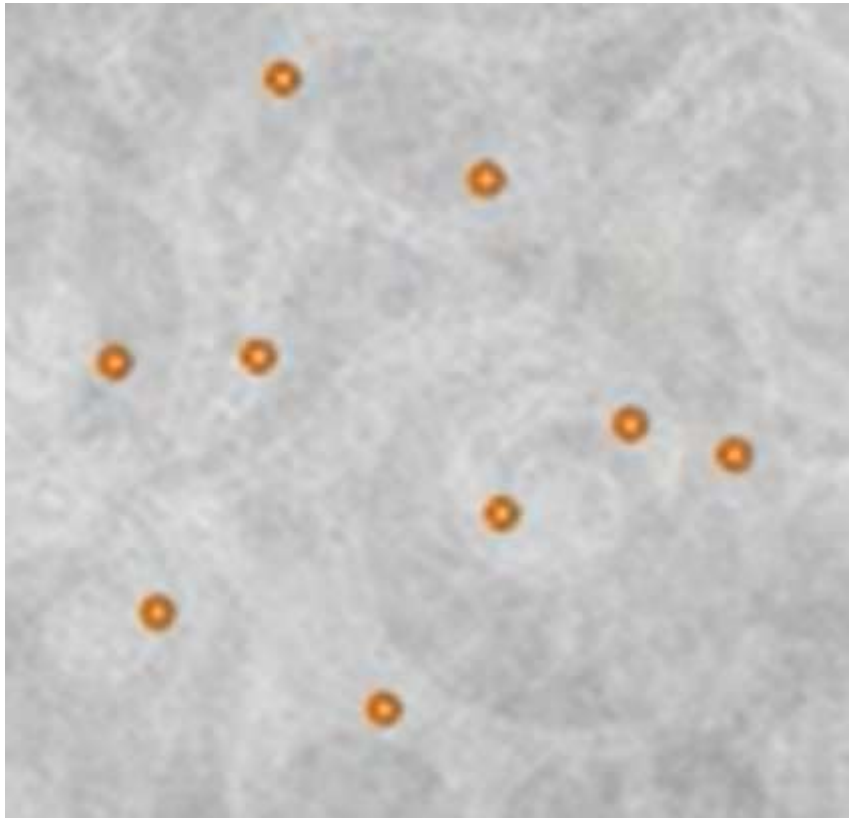
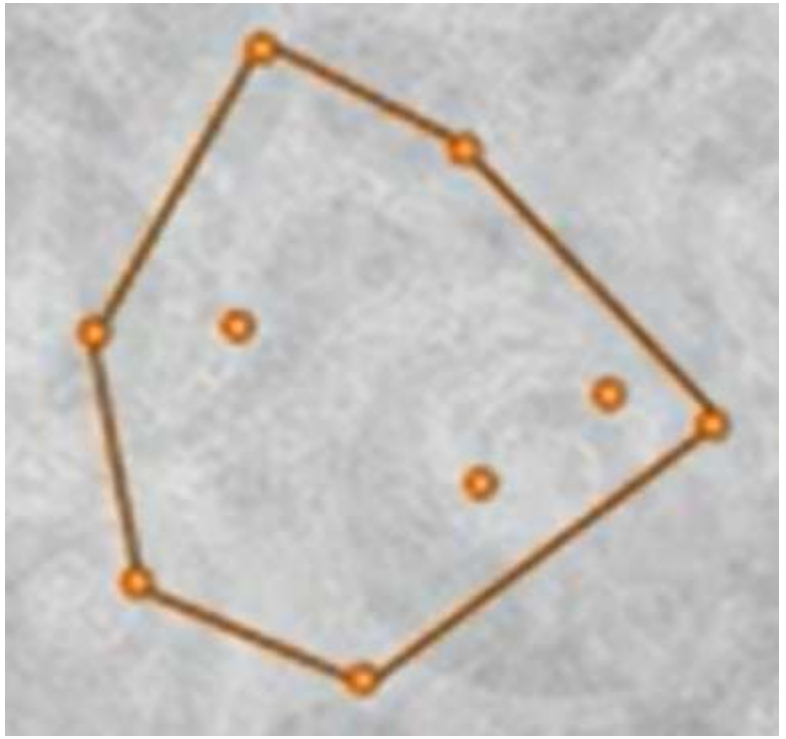
Convex Hull

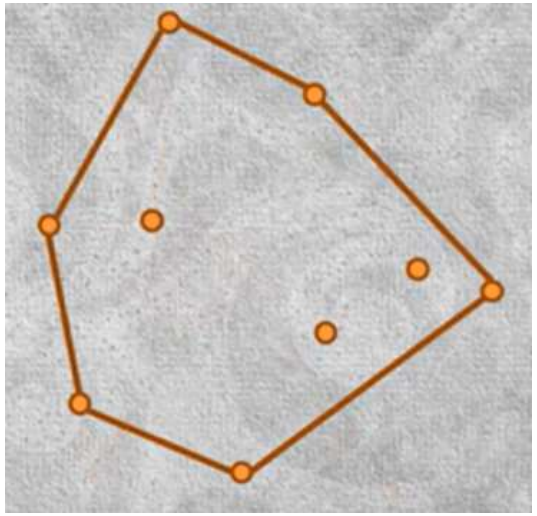
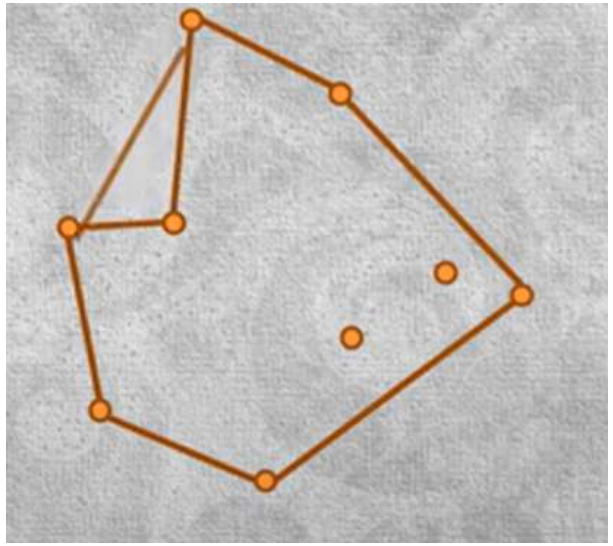
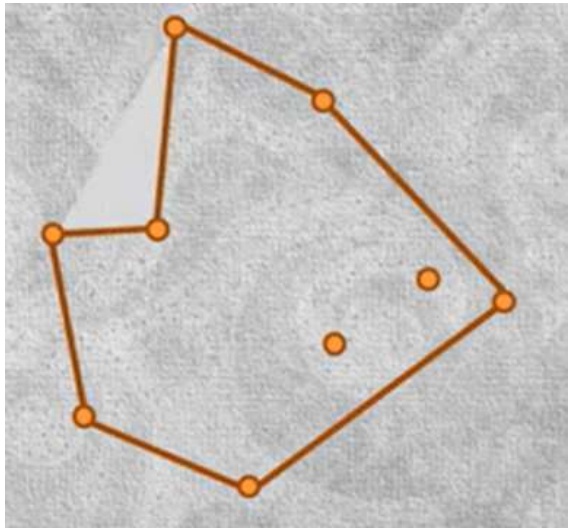
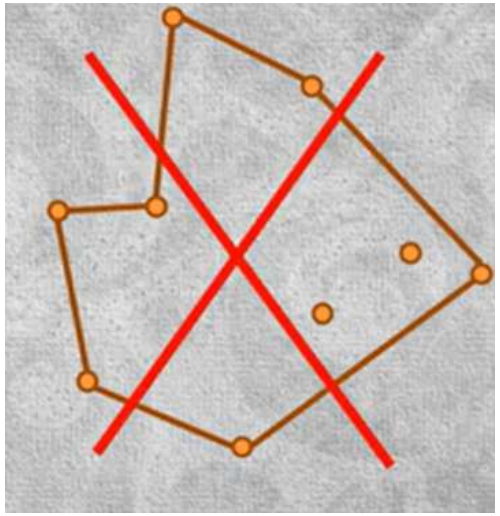
- Given a set of points on a 2-dimensional plane, a convex hull is a geometric object, a polygon that encloses all of those points.
- The vertices of this polygon maximize the area while minimizing the circumference.



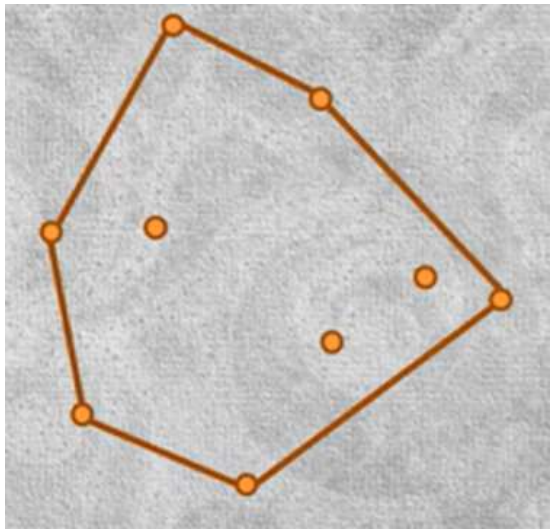
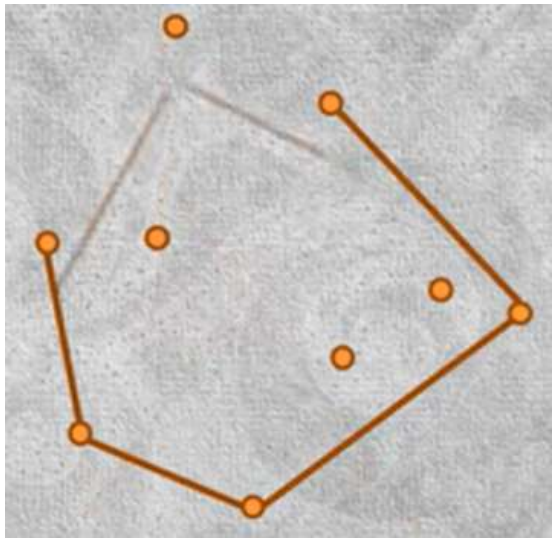
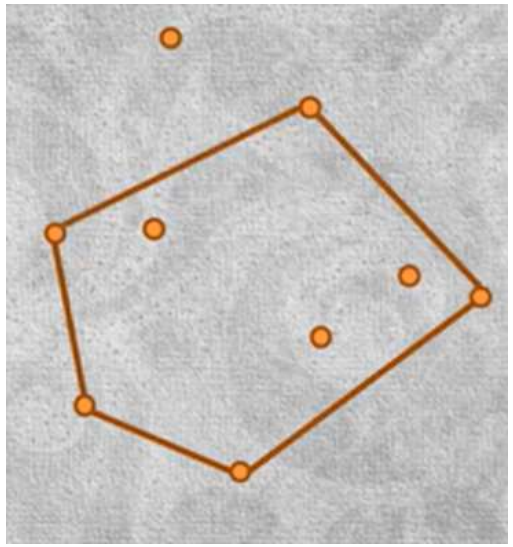
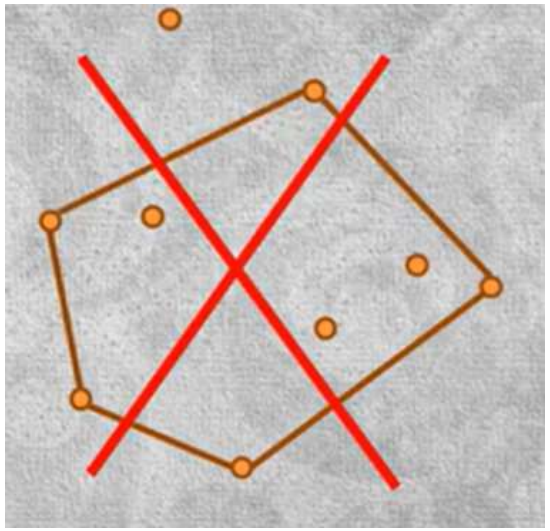
- Given a set of points on a 2-dimensional plane, a convex hull is a geometric object, a polygon, that encloses all of those points.
- The vertices of this polygon maximize the area while minimizing the circumference.
- Here(in the figure) is the convex hull of particular set of points.



- **Note that if some additional points were to be included, then the covering area is reduced while the circumference is increased.**
- Likewise, if some of the vertices of the Convex Hull are omitted, then the resulting polygon won't cover all of the points in the set.

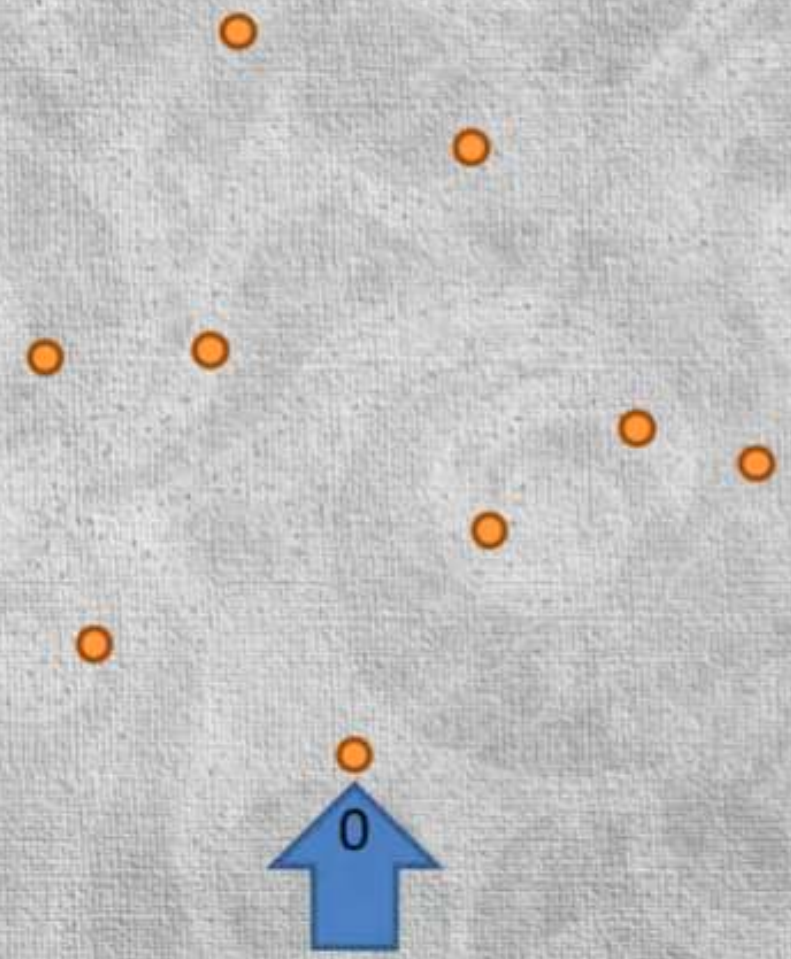
- Note that if some additional points were to be included, then the covering area is reduced while the circumference is increased.
- Likewise, if some of the vertices of the Convex Hull are omitted, then the resulting polygon won't cover all of the points in the set.

- The Convex Hull has a **wide variety of applications**, ranging from image recognition in robotics to determining animal's home range in ethology.
- To calculate the Convex Hull using two different algorithms.
  - The first one is called "**Graham Scan**" while the second is called "**Jarvis March**", also known as "gift wrapping algorithm".
  - Both of them are **conceptually fairly simple**, but there are a few tricky implementation pitfalls that will be seen here.

Graham scan algorithm

select the point with the lowest Y coordinate



Graham Scan algorithm **starts** by first looping over all of the points to **select** the one with the lowest Y coordinate.

This is the bottom most point and, assuming **no collinear points**, is therefore guaranteed to be one of the vertices of the Convex Hull.

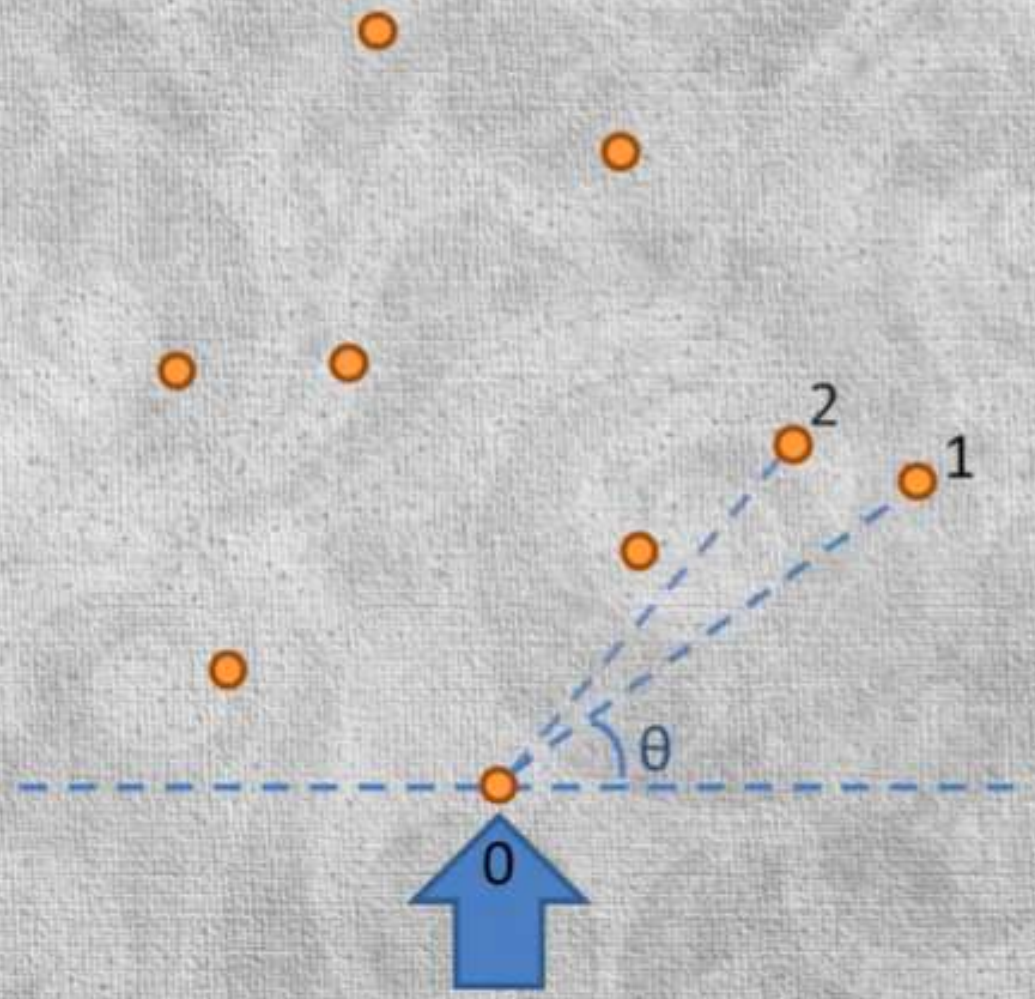
Graham scan algorithm

select the point with the lowest Y coordinate

Graham Scan algorithm **starts** by first looping over all of the points to **select** the one with the lowest Y coordinate.

it could just as well pick the top most or the leftmost or the rightmost point, but by **convention** it **picks** the bottom most.

Graham scan algorithm



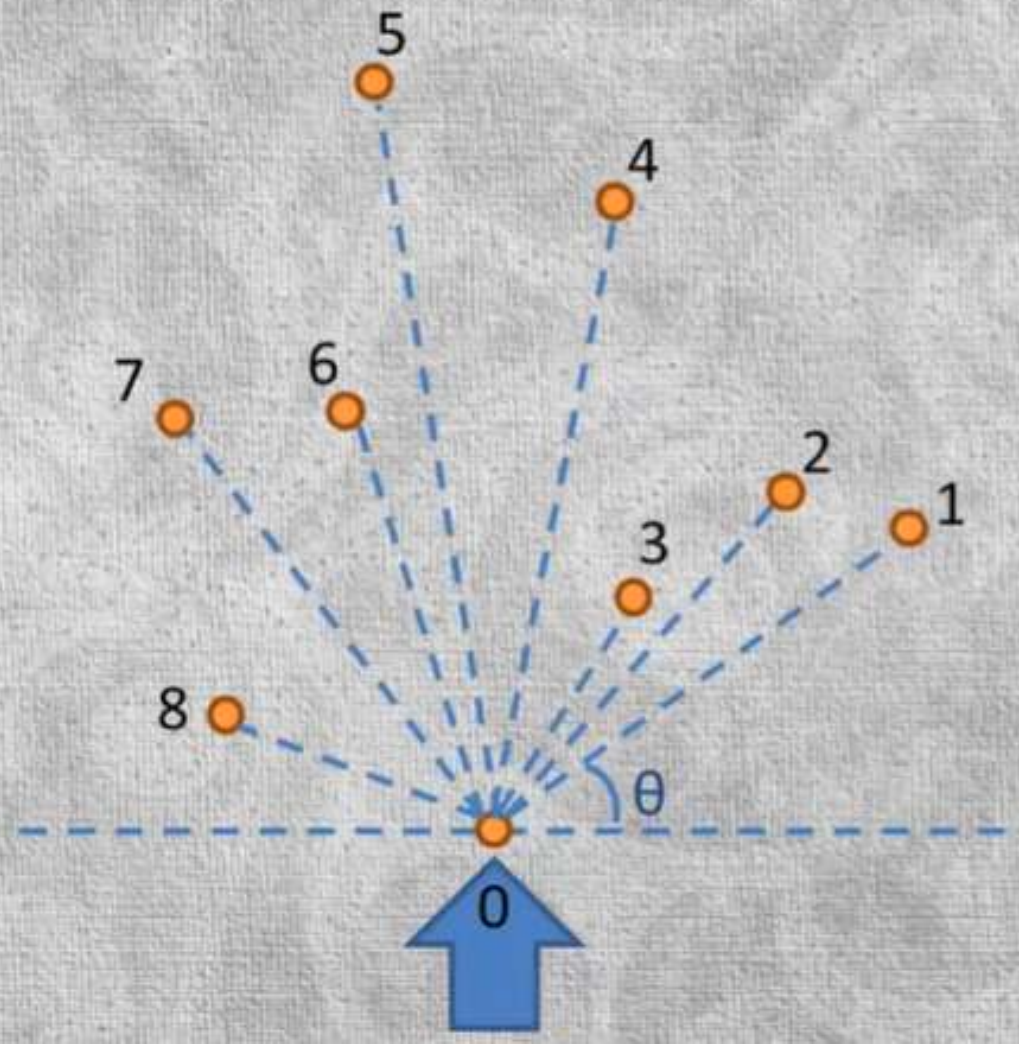
select the point with the lowest Y coordinate

sort the points by the angle relative to the bottom most point and the horizontal

Then it **sorts** the remaining points, **ordering them** by the **angle** that they **make relative** to the **bottom most point** and the **horizontal**.

You can see that the **angles** would range from **0** to **180** degrees, given that the **reference point** is the **bottom most one** in the set.

Graham scan algorithm



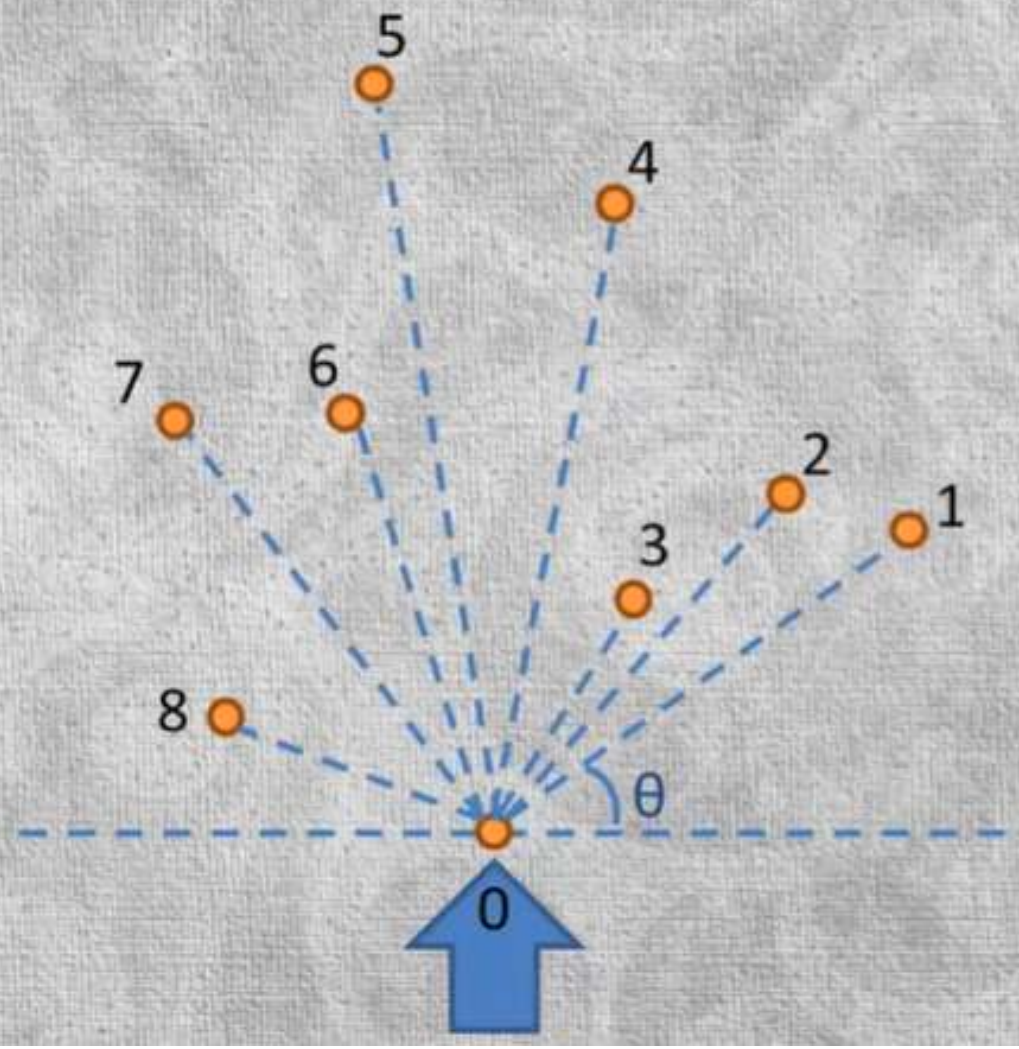
select the point with the lowest Y coordinate

sort the points by the angle relative to the bottom most point and the horizontal

Then it **sorts** the remaining points, **ordering them** by the **angle** that they **make relative** to the **bottom most point** and the **horizontal**.

You can see that the **angles** would range from **0** to **180** degrees, given that the **reference point** is the **bottom most one** in the set.

Graham scan algorithm

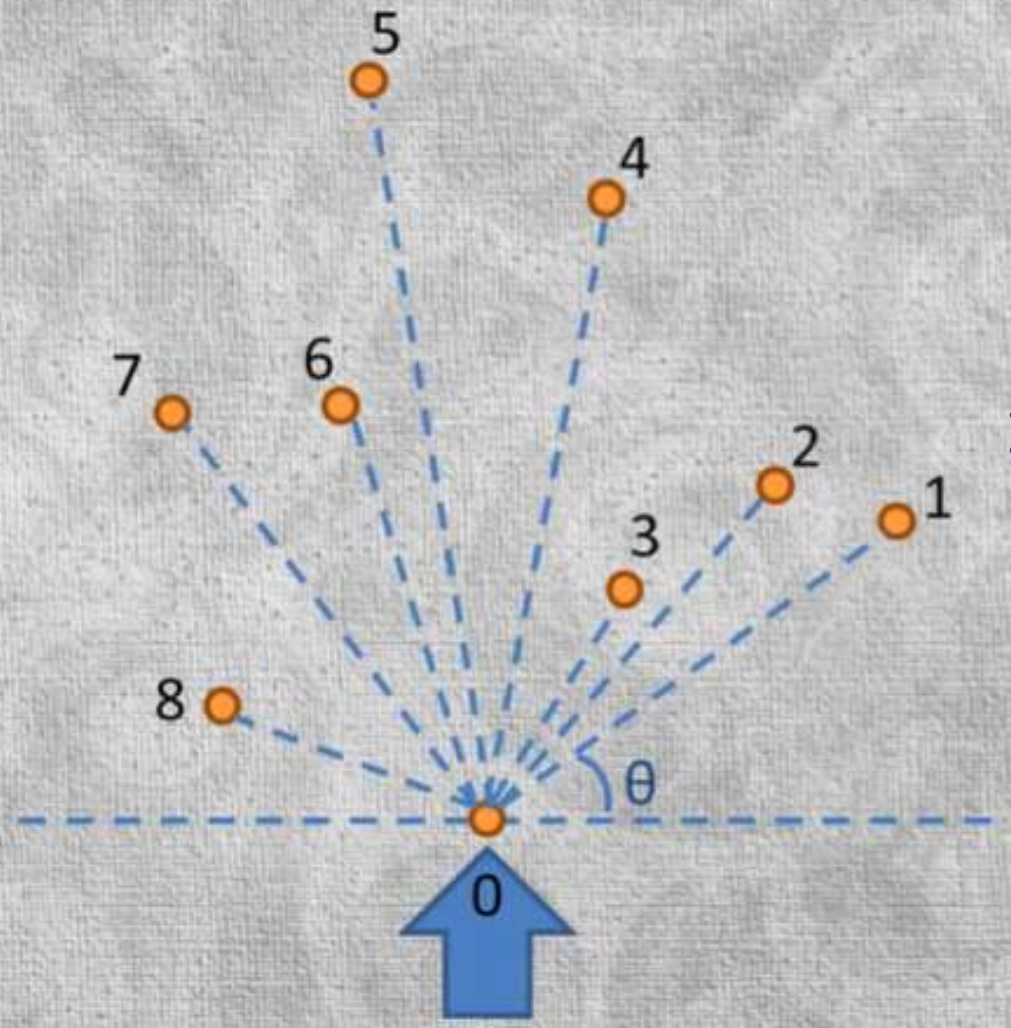


select the point with the lowest Y coordinate

sort the points by the angle relative to the bottom most point and the horizontal

Finally, the algorithm loops over those points in **sorted order**, effectively scanning in the **counterclockwise direction**. **Each point** is placed on a **stack**, but **only** if it **makes a line** with a **counterclockwise** turn relative to the previous 2 points on the stack.

Graham scan algorithm



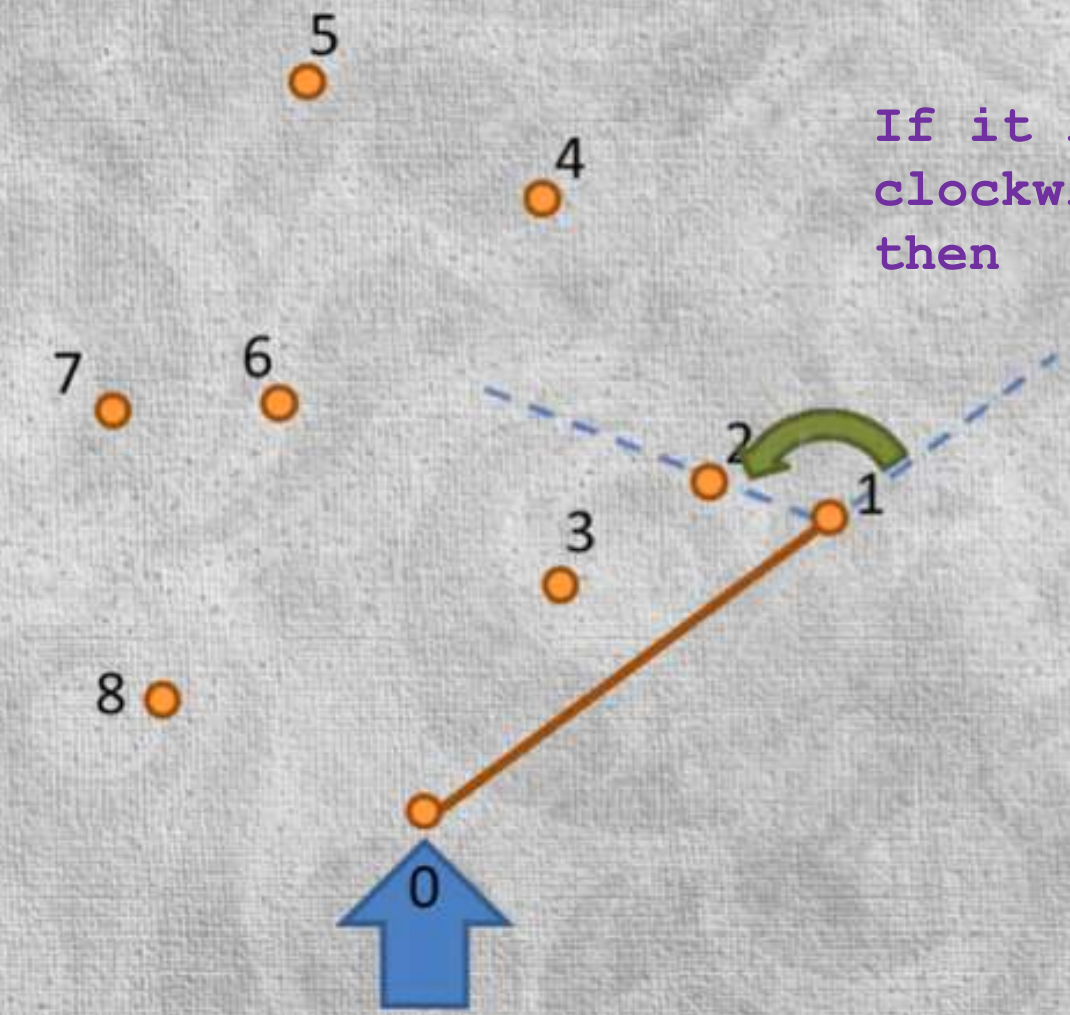
select the point with the lowest Y coordinate

sort the points by the angle relative to the bottom most point and the horizontal

Finally, iterate in sorted order, placing each point on a stack, but only if it makes a **counterclockwise** turn relative to the previous 2 points on the stack

If it is clockwise, then pop previous point off of the stack if making a **clockwise** turn

Graham scan algorithm



iterate in sorted order, placing each point on a stack, but only if it makes a **counterclockwise** turn relative to the previous 2 points on the stack

`If it is clockwise, then`

pop previous point off of the stack if making a **clockwise** turn

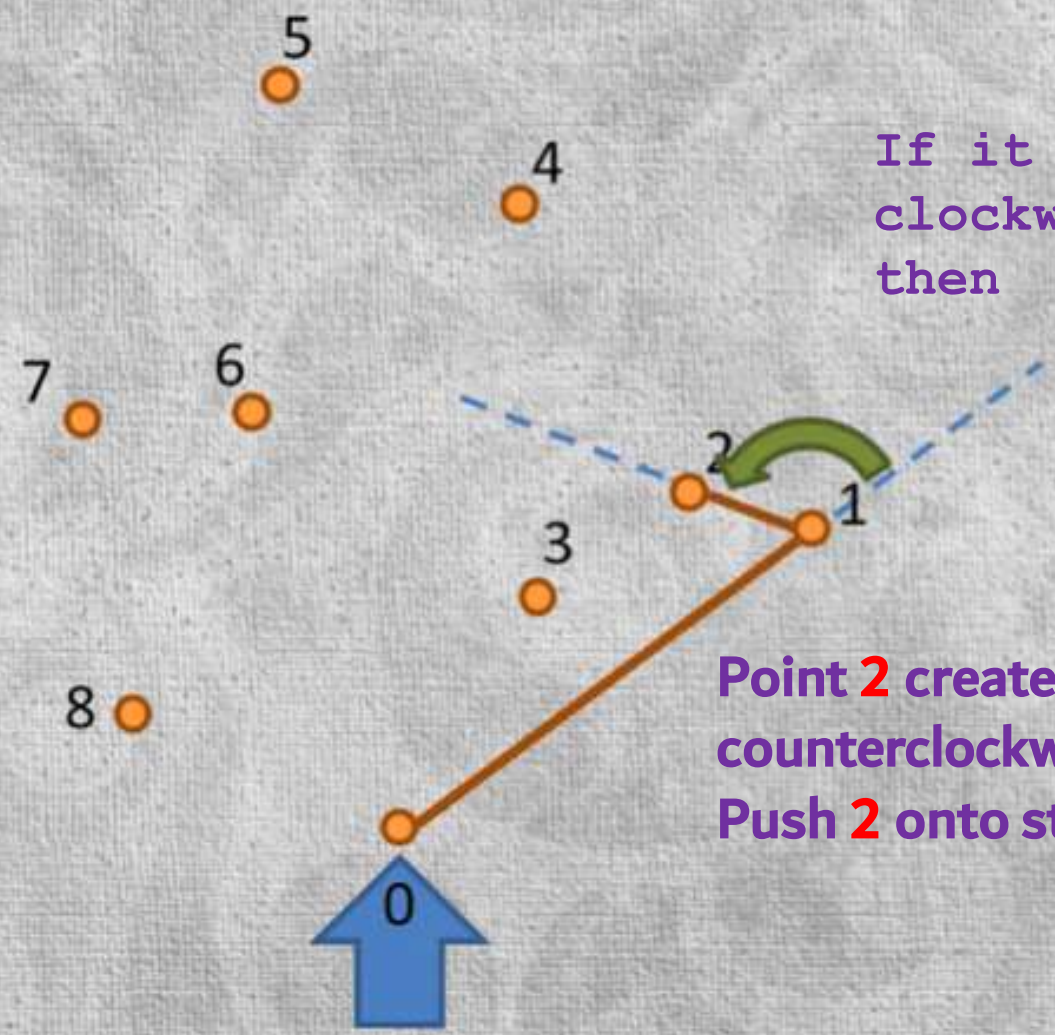
Each point is placed on a **stack**, but **only** if it **makes a line** with a **counterclockwise turn** relative to the previous 2 points on the stack.

Graham scan algorithm

iterate in sorted order, placing each point on a stack, but only if it makes a **counterclockwise** turn relative to the previous 2 points on the stack

`If it is clockwise, then`

pop previous point off of the stack if making a **clockwise** turn



Point 2 creates: counterclockwise
Push 2 onto stack

The algorithm continues. **Pushing** points if found counterclockwise **or** **popping** points off the stack until the resulting turn is counterclockwise.

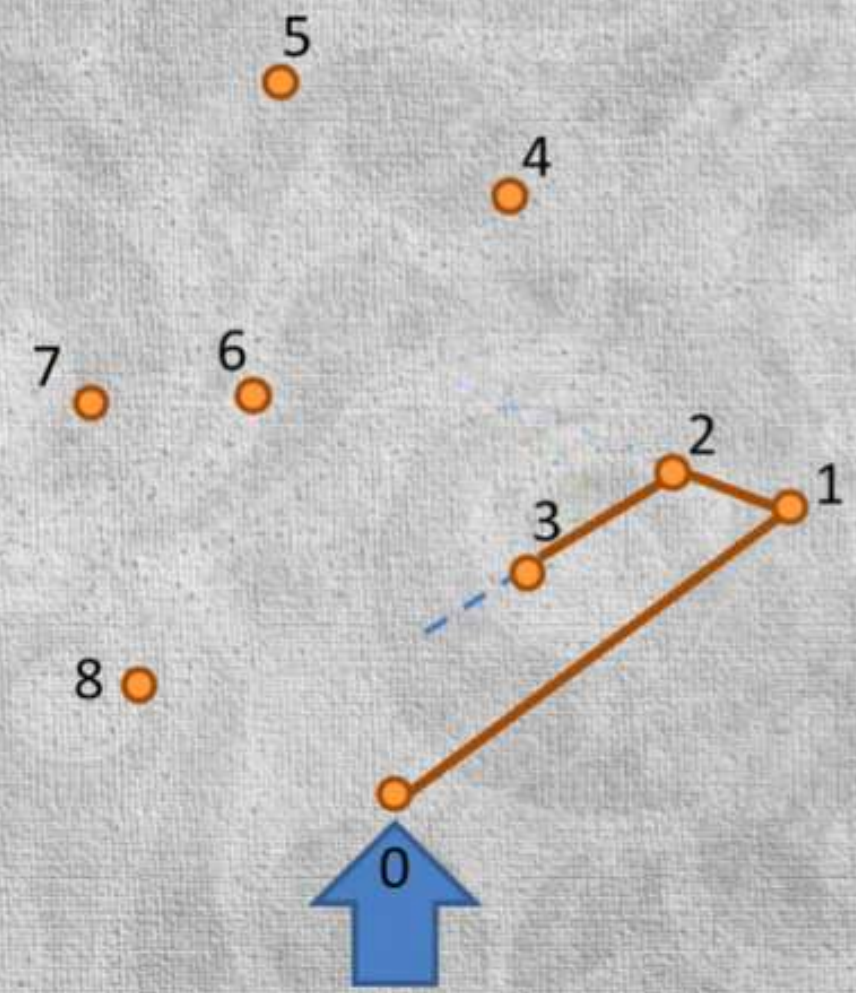
Graham scan algorithm

iterate in sorted order, placing each point on a stack, but only if it makes a **counterclockwise** turn relative to the previous 2 points on the stack

pop previous point off of the stack if making a **clockwise** turn

Point 3 creates: counterclockwise
Push 3 onto stack

Graham scan algorithm



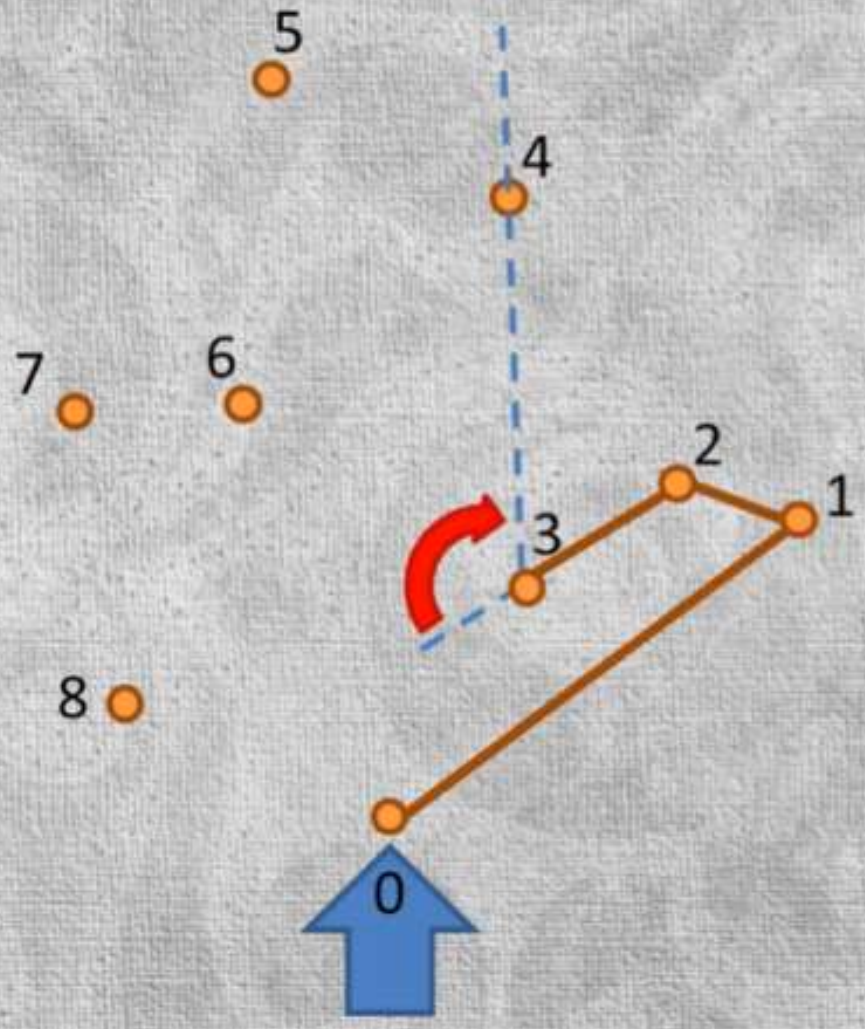
iterate in sorted order, placing each point on a stack, but only if it makes a **counterclockwise** turn relative to the previous 2 points on the stack

pop previous point off of the stack if making a **clockwise** turn

In this example, Points 2 and 3 are placed in Stack B'coz at that time, the connecting lines were making **counter-clockwise turns.**

Stable Sort

Graham scan algorithm



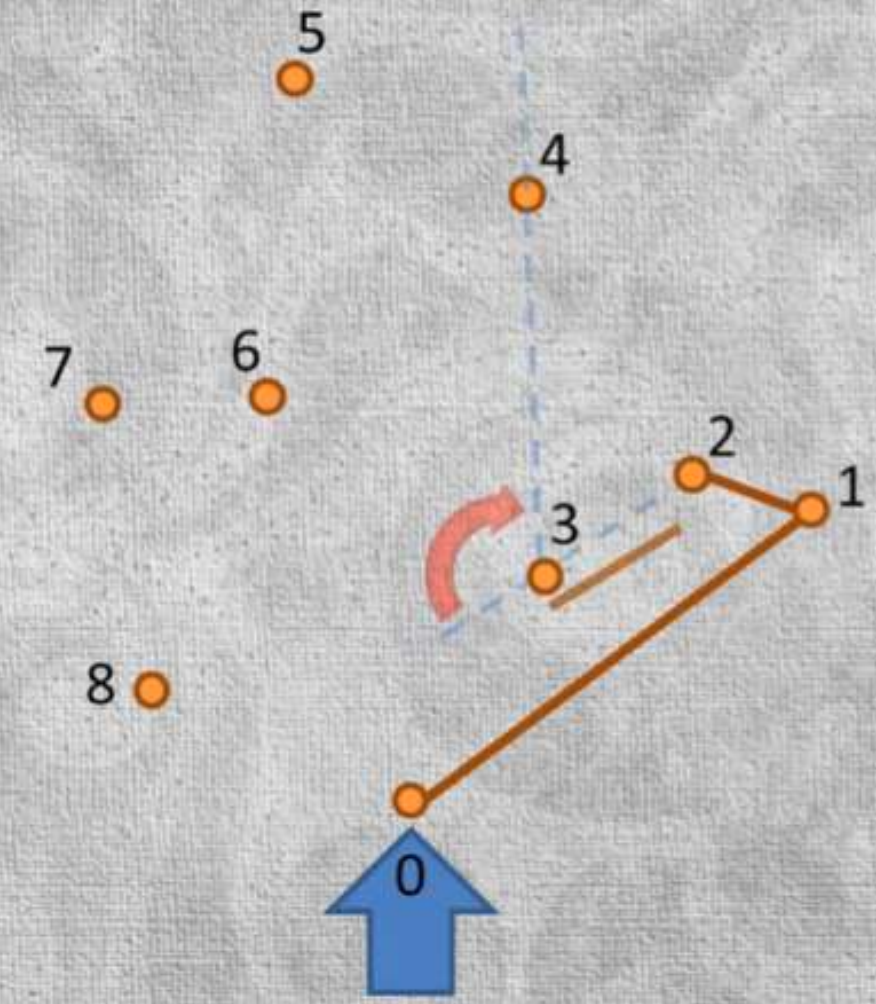
iterate in sorted order, placing each point on a stack, but only if it makes a **counterclockwise** turn relative to the previous 2 points on the stack

pop previous point off of the stack if making a **clockwise** turn

But Point 4 creates clockwise turn.

Stable Sort

Graham scan algorithm



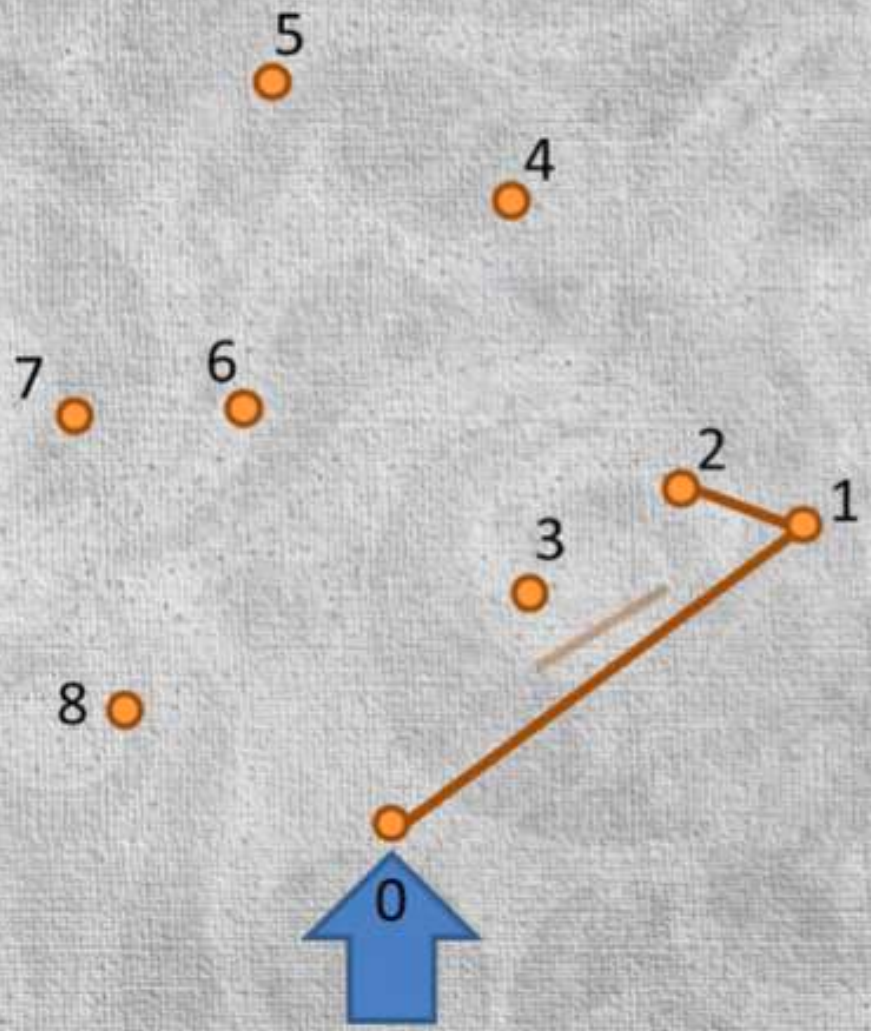
iterate in sorted order, placing each point on a stack, but only if it makes a **counterclockwise** turn relative to the previous 2 points on the stack

pop previous point off of the stack if making a **clockwise** turn

But Point 4 creates **clockwise** turn.
So, we **rollback, popping** off point 3

Stable Sort

Graham scan algorithm



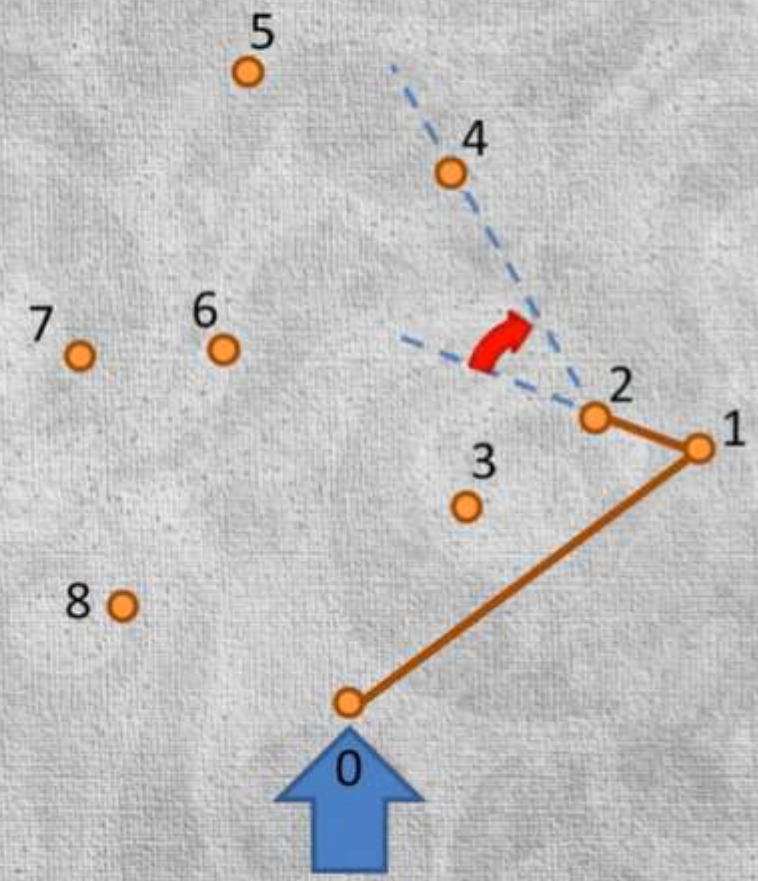
iterate in sorted order, placing each point on a stack, but only if it makes a **counterclockwise** turn relative to the previous 2 points on the stack

pop previous point off of the stack if making a **clockwise** turn

But Point 4 creates **clockwise** turn.
So, we **rollback, popping** off point **3**

Stable Sort

Graham scan algorithm



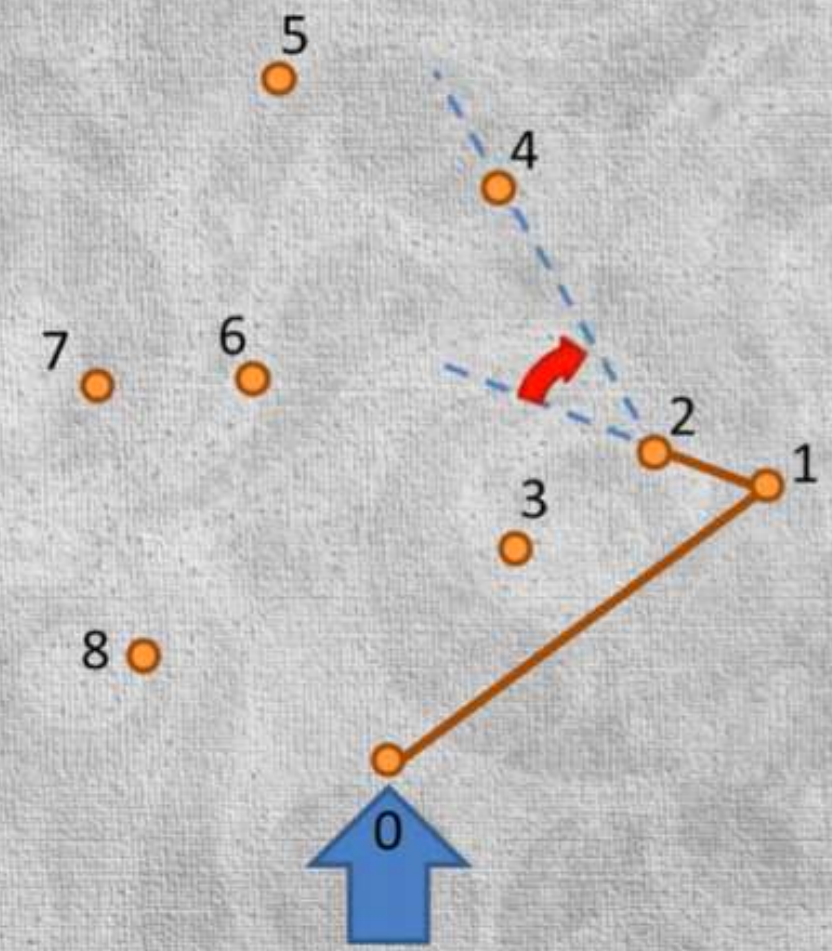
iterate in sorted order, placing each point on a stack, but only if it makes a **counterclockwise** turn relative to the previous 2 points on the stack

pop previous point off of the stack if making a **clockwise** turn

But Point 4 creates clockwise turn.
So, we **rollback, popping** off point **3**
And then point 2: creates **clockwise** turn

Stable Sort

Graham scan algorithm



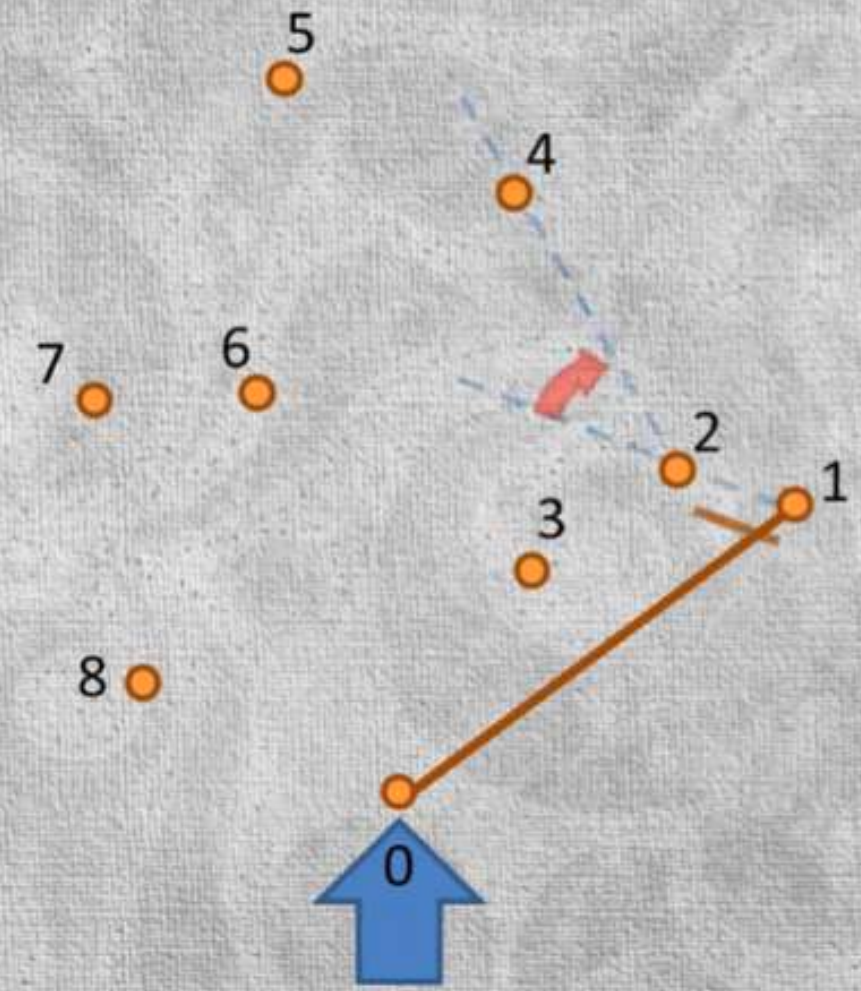
iterate in sorted order, placing each point on a stack, but only if it makes a **counterclockwise** turn relative to the previous 2 points on the stack

pop previous point off of the stack if making a **clockwise** turn

And then point 2: creates clockwise turn
So, again **rollback, Pop 2**

Stable Sort

Graham scan algorithm



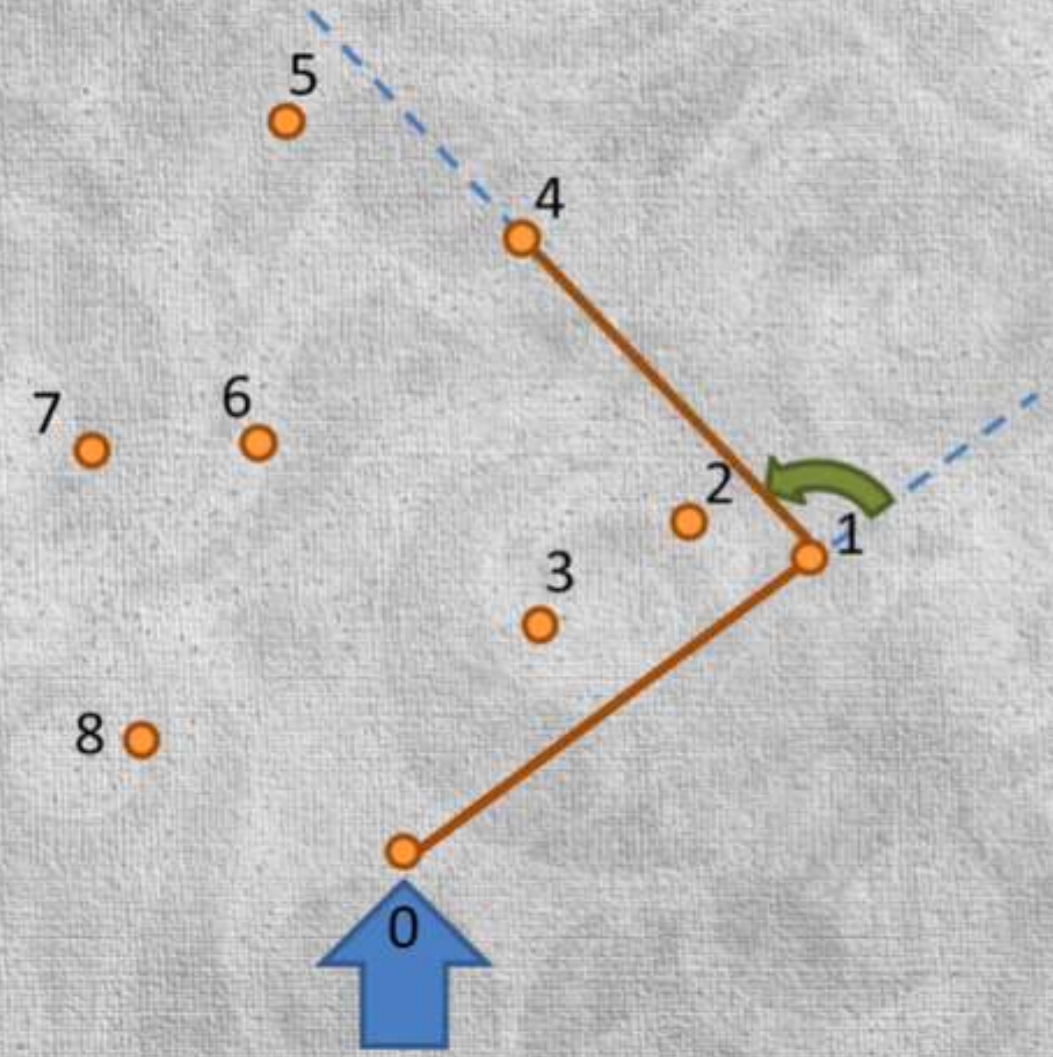
iterate in sorted order, placing each point on a stack, but only if it makes a **counterclockwise** turn relative to the previous 2 points on the stack

pop previous point off of the stack if making a **clockwise** turn

And then point 2: creates clockwise turn
So, again **rollback, Pop 2**
and until the resulting line makes counter clockwise **turn**

Stable Sort

Graham scan algorithm



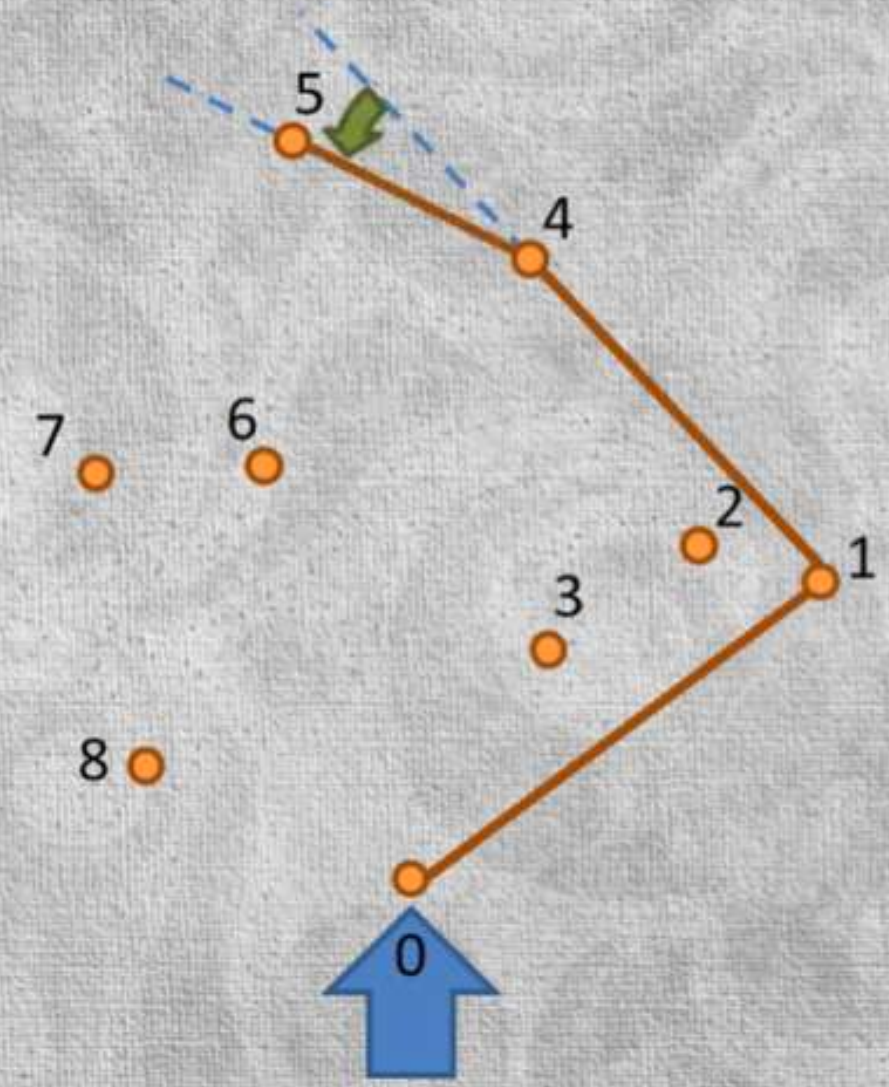
iterate in sorted order, placing each point on a stack, but only if it makes a **counterclockwise** turn relative to the previous 2 points on the stack

pop previous point off of the stack if making a **clockwise** turn

Point 4 makes counter clockwise turn. So push 4 onto stack

Stable Sort

Graham scan algorithm

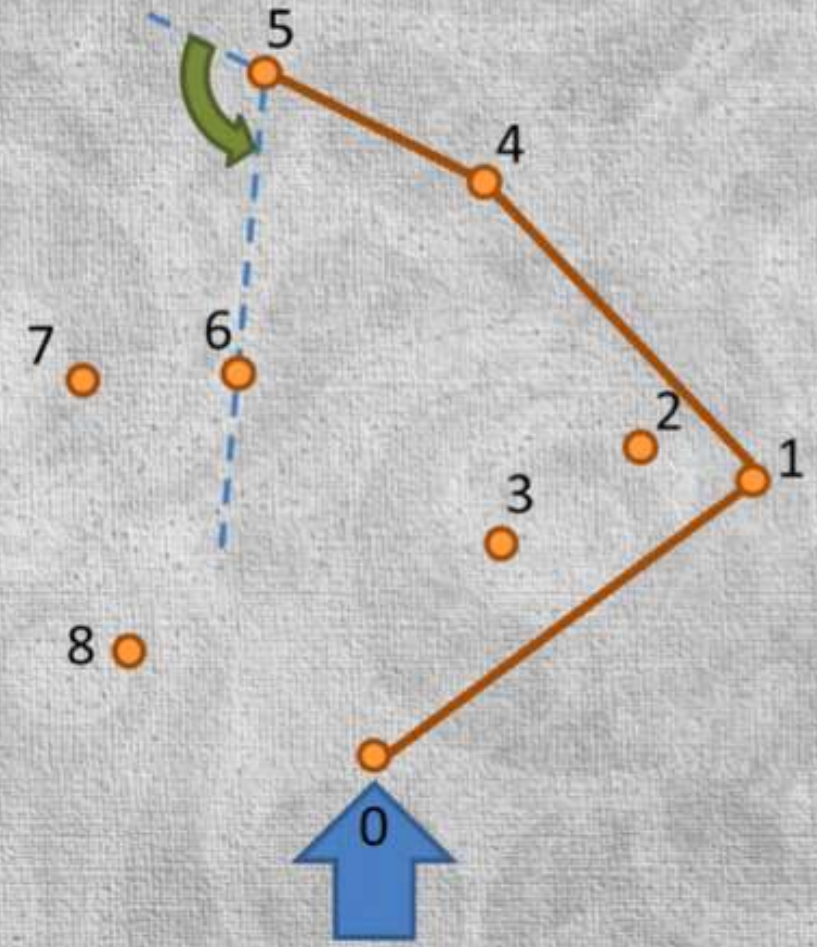


iterate in sorted order, placing each point on a stack, but only if it makes a **counterclockwise** turn relative to the previous 2 points on the stack

pop previous point off of the stack if making a **clockwise** turn

Point 5 also makes counter clockwise turn. So push 5 onto stack

Graham scan algorithm

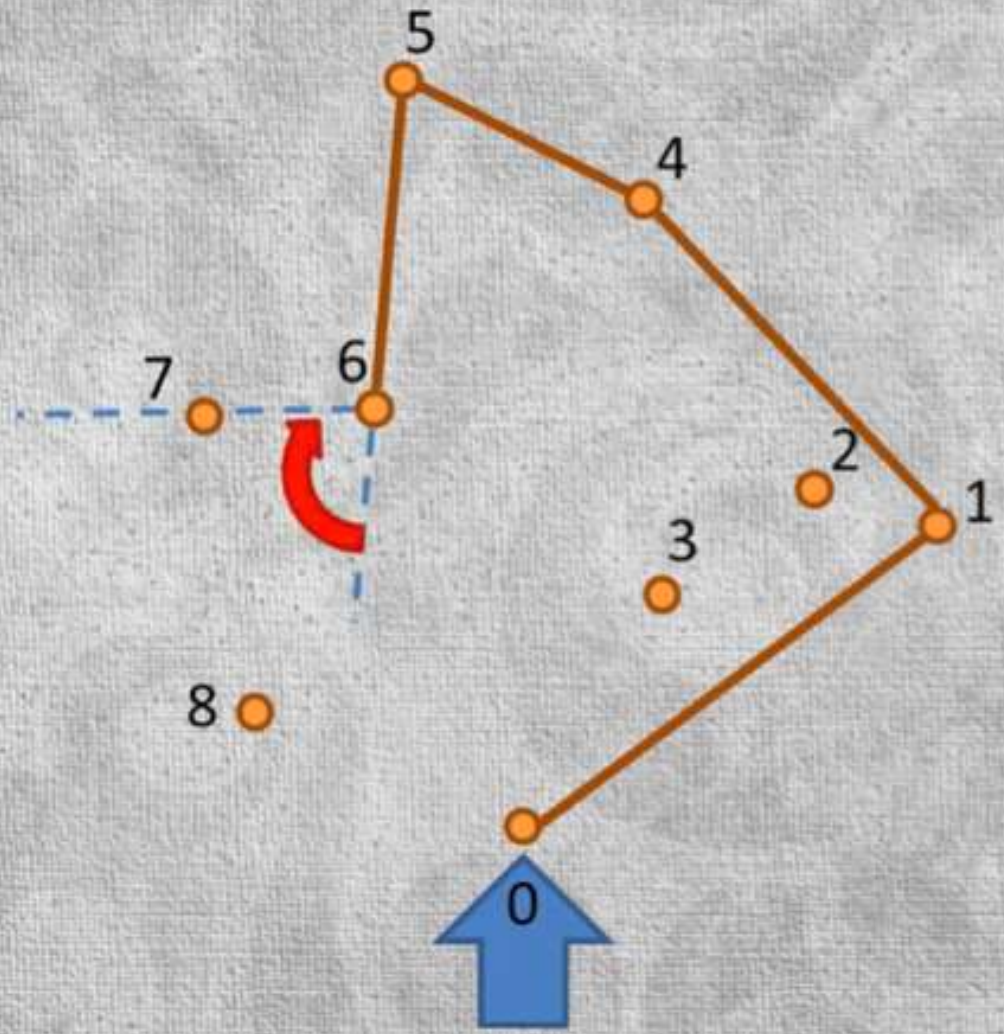


iterate in sorted order, placing each point on a stack, but only if it makes a **counterclockwise** turn relative to the previous 2 points on the stack

pop previous point off of the stack if making a **clockwise** turn

Point **6** also makes counter clockwise turn. So push **6** onto stack, sense it could potentially be part of the convex hull. But we don't know yet.

Graham scan algorithm

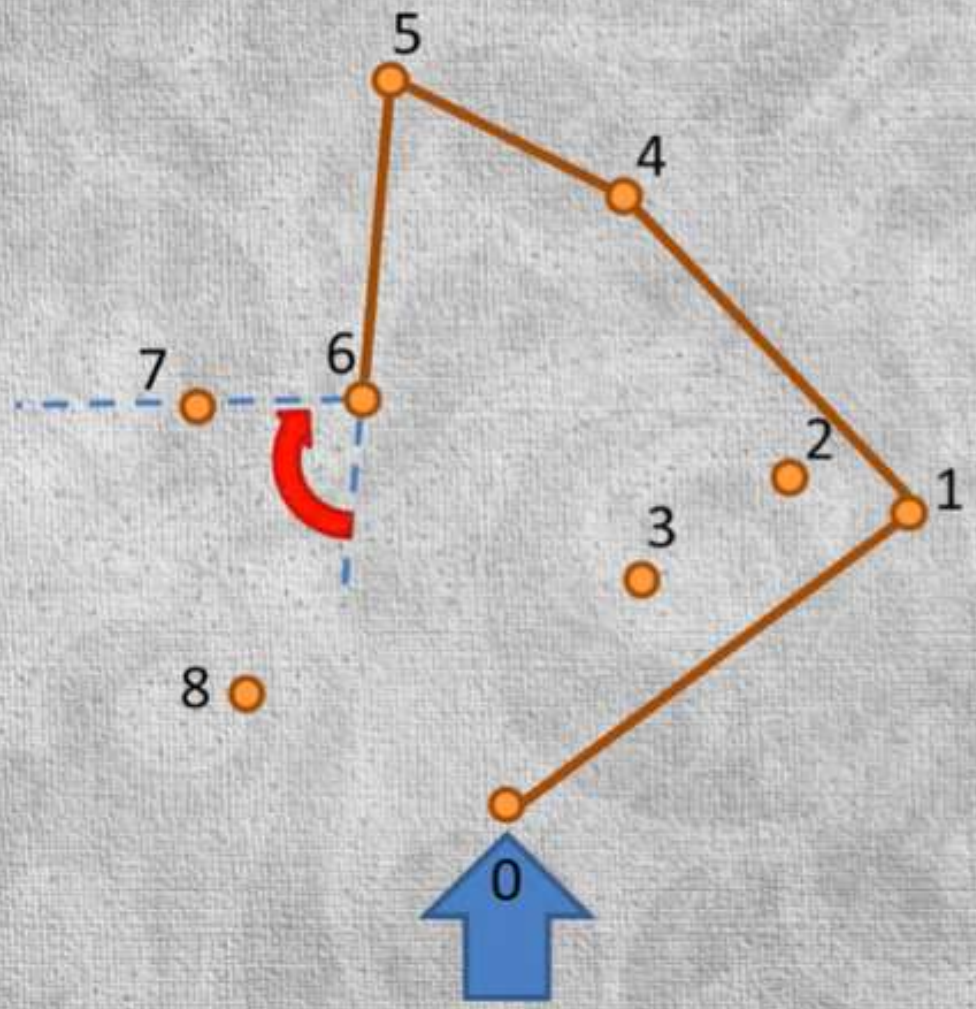


iterate in sorted order, placing each point on a stack, but only if it makes a **counterclockwise** turn relative to the previous 2 points on the stack

pop previous point off of the stack if making a **clockwise** turn

Point 7 makes clockwise turn. So **rollback**, pop out 6

Graham scan algorithm



iterate in sorted order, placing each point on a stack, but only if it makes a **counterclockwise** turn relative to the previous 2 points on the stack

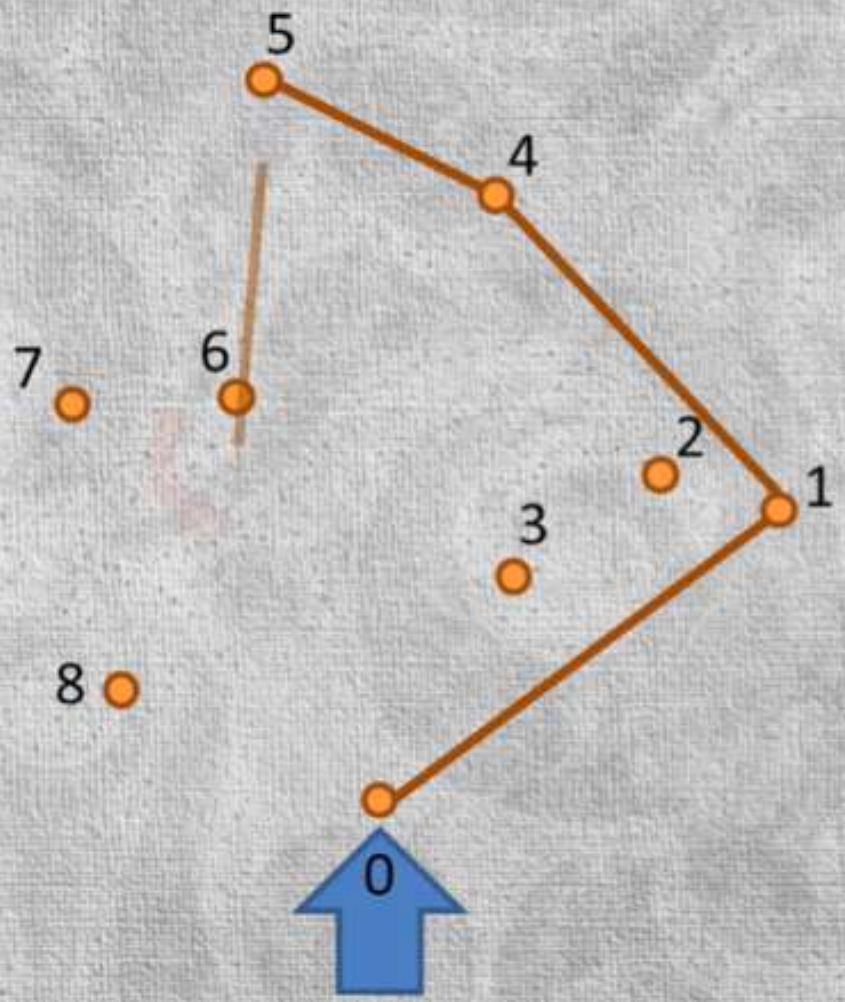
pop previous point off of the stack if making a **clockwise** turn

Point **7** makes clockwise turn. So rollback, **pop** out **6**

i.e. Remove 6 line

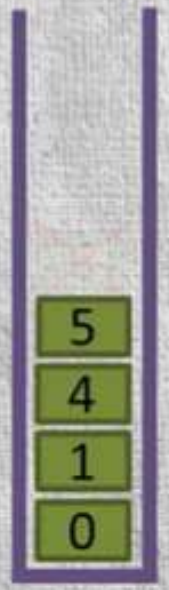
Stable Sort

Graham scan algorithm



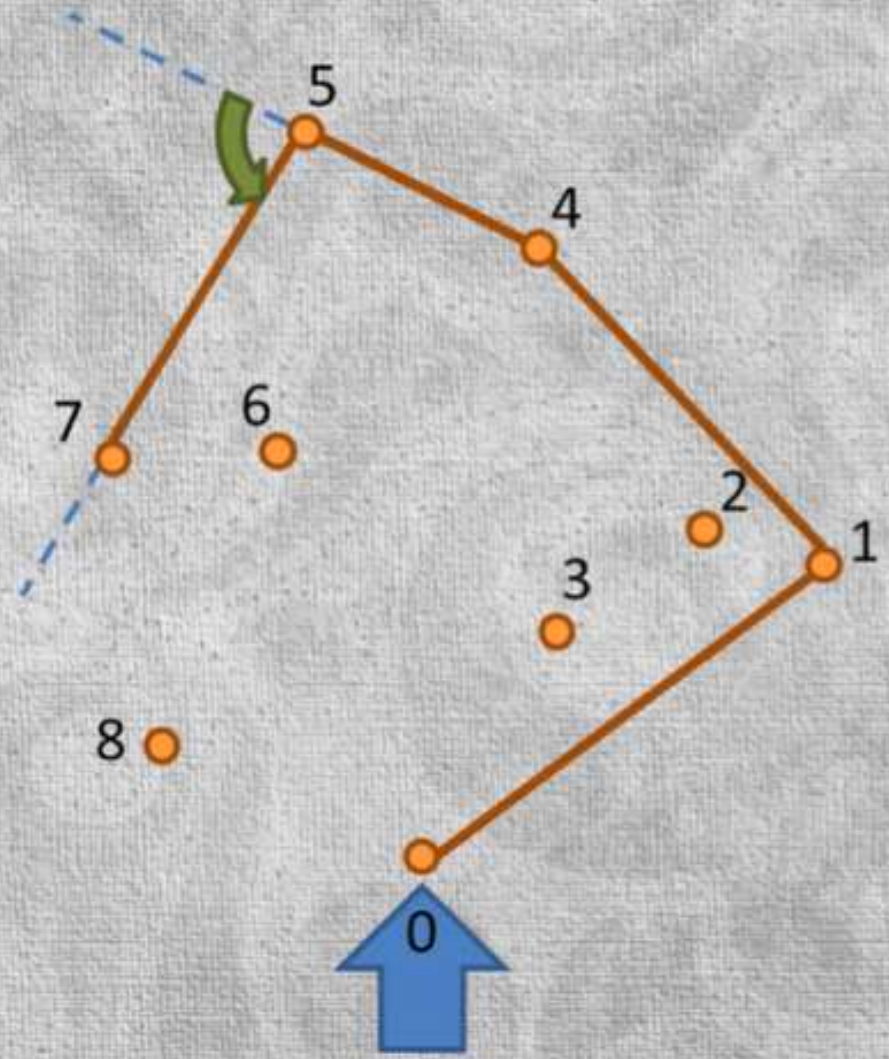
iterate in sorted order, placing each point on a stack, but only if it makes a **counterclockwise** turn relative to the previous 2 points on the stack

pop previous point off of the stack if making a **clockwise** turn



Next point is **5**

Graham scan algorithm



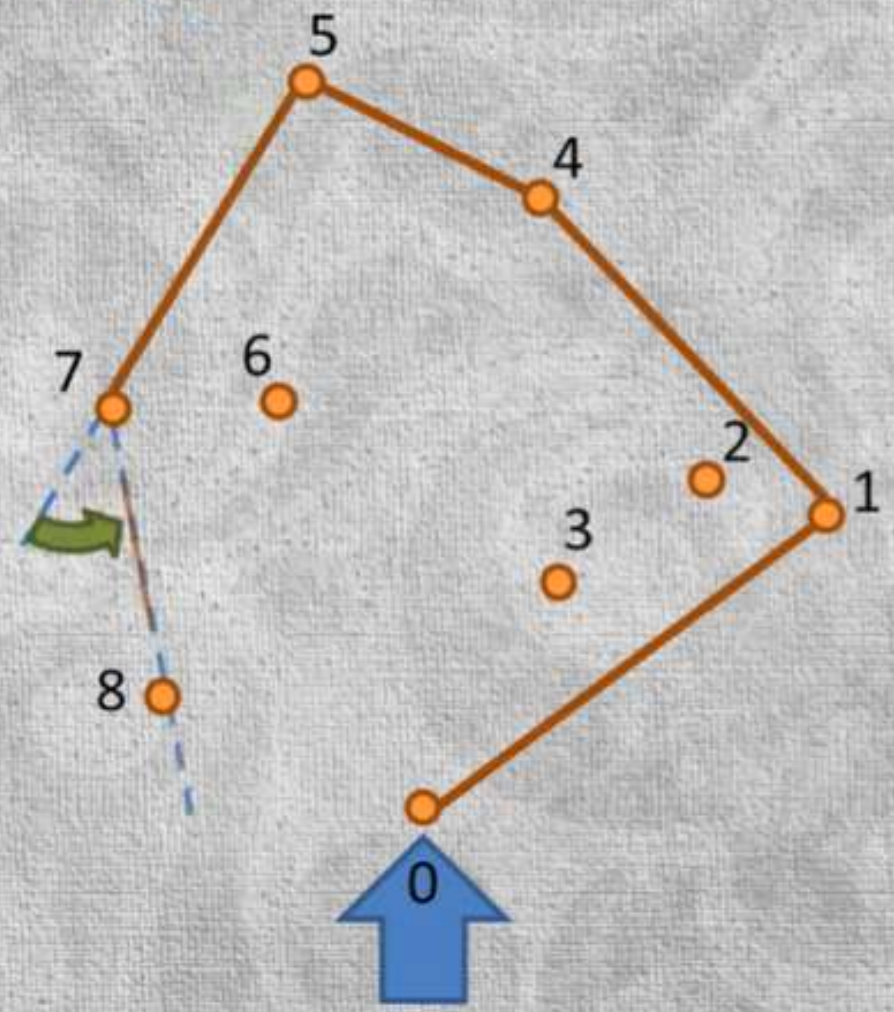
iterate in sorted order, placing each point on a stack, but only if it makes a **counterclockwise** turn relative to the previous 2 points on the stack

pop previous point off of the stack if making a **clockwise** turn

Point 7 creates: counter clockwise turn. So push 7 onto stack,

Stable Sort

Graham scan algorithm



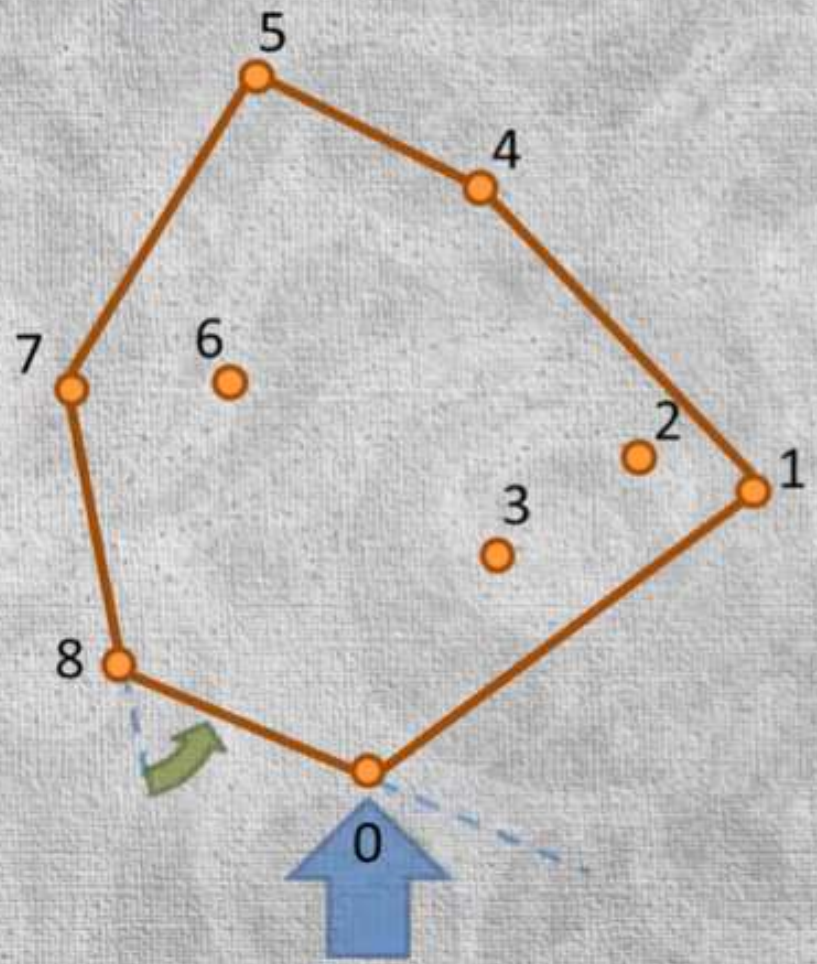
iterate in sorted order, placing each point on a stack, but only if it makes a **counterclockwise** turn relative to the previous 2 points on the stack

pop previous point off of the stack if making a **clockwise** turn

Point **8** creates: counter clockwise turn. So push **8** onto stack,

Stable Sort

Graham scan algorithm



iterate in sorted order, placing each point on a stack, but only if it makes a **counterclockwise** turn relative to the previous 2 points on the stack

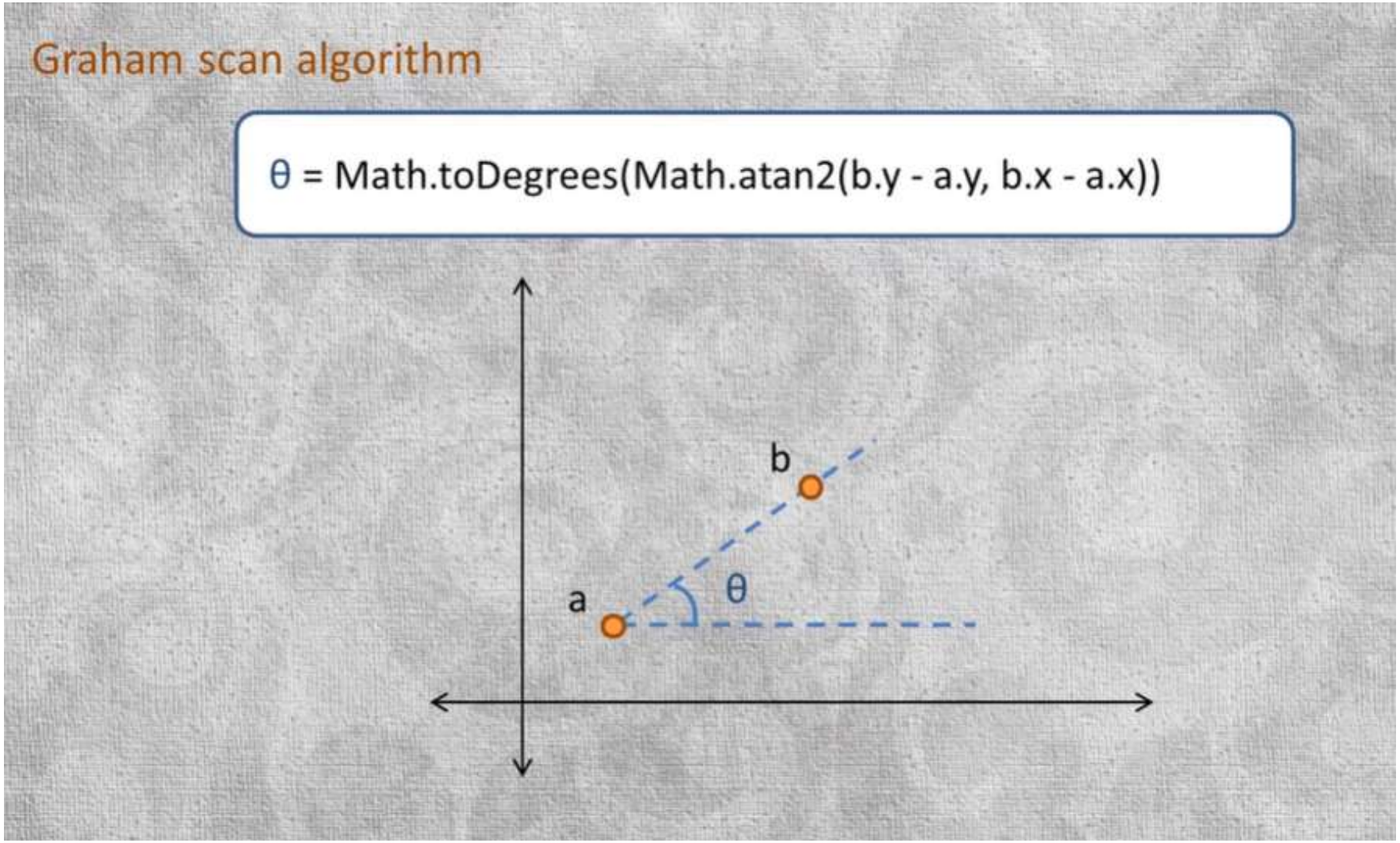
pop previous point off of the stack if making a **clockwise** turn

Keep continuing until you get back to the **starting point.**

Stable Sort

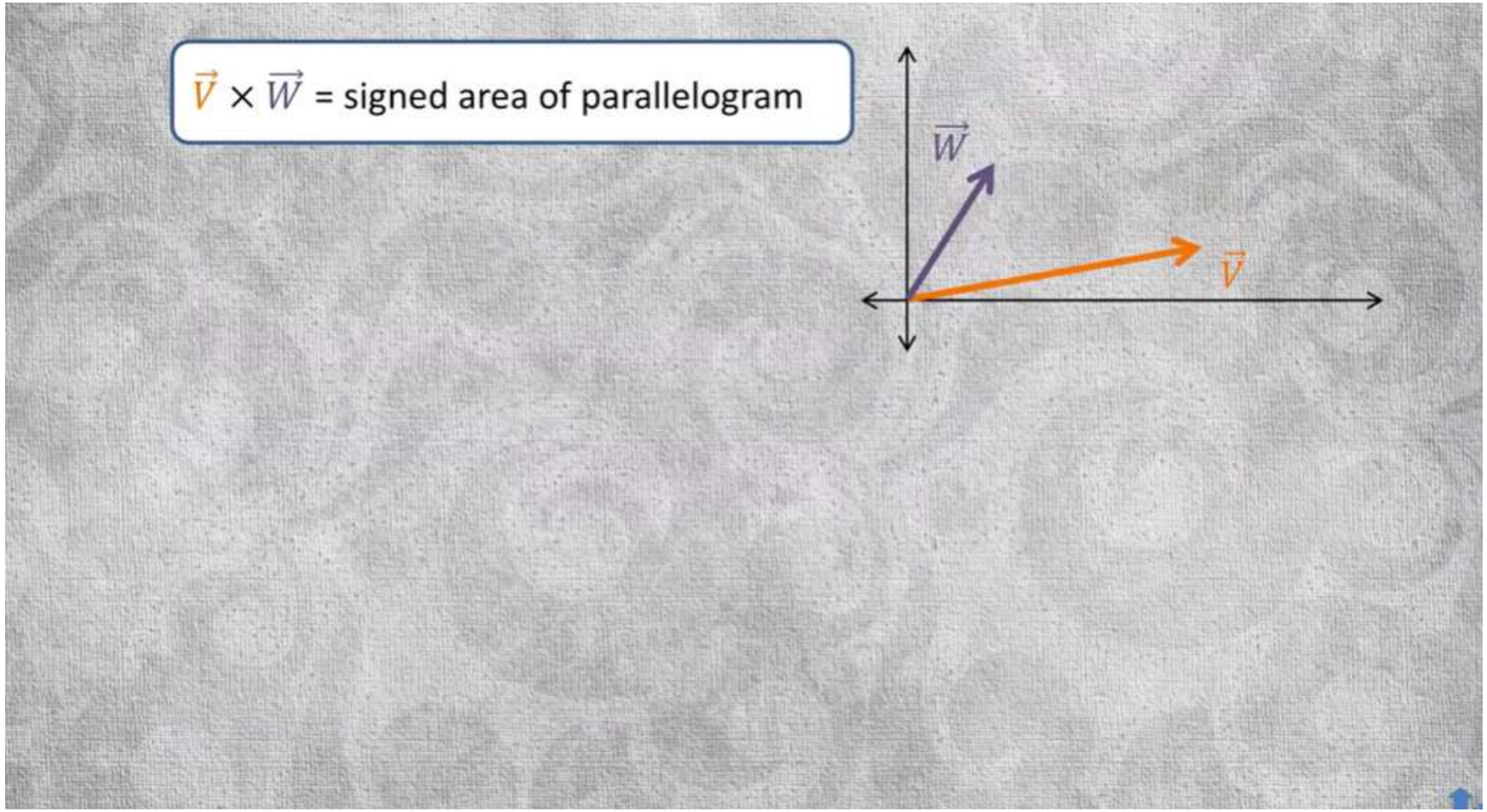
Implementation Details

- To determine clockwise turn or not, you could use trigonometry angles in degrees and then compare them.
- But we **don't really care** what the actual angles are
- We just wants to know if its



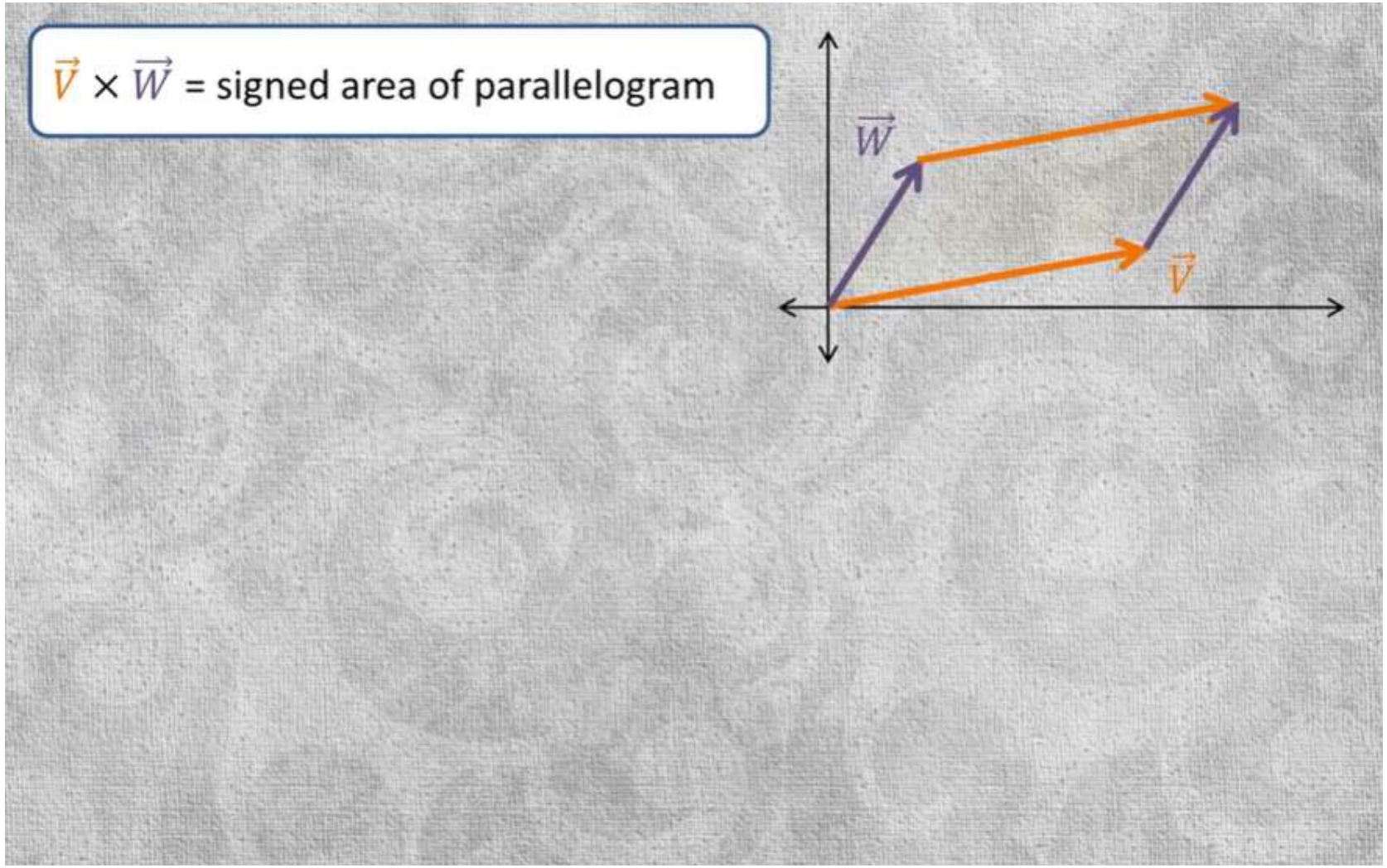
Implementation Details

- Let V and W are 2 vectors
- The cross product of 2 vectors V and W calculates the signed area of parallelogram that is formed if you connect the 2 vectors like parallelogram



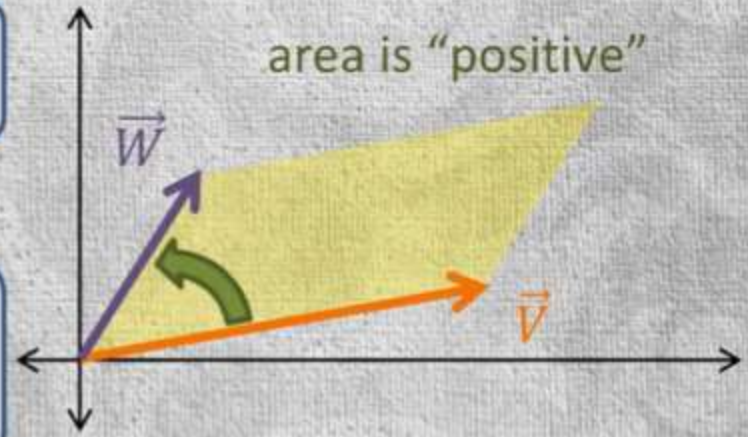
Implementation Details

- Let V and W are 2 vectors
- The cross product of 2 vectors V and W calculates the signed area of parallelogram that is formed if you connect the 2 vectors like parallelogram

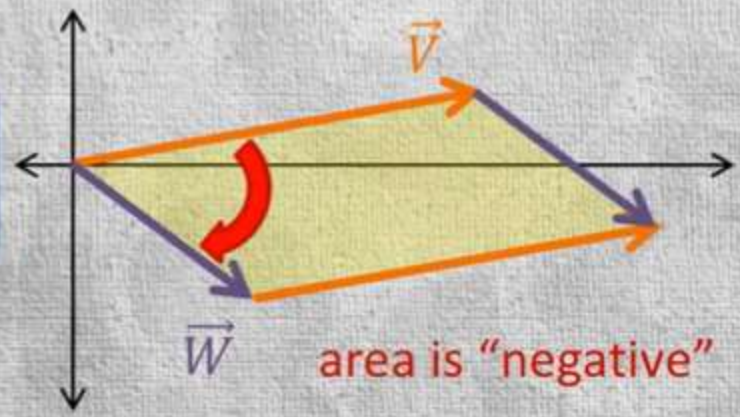


$\vec{V} \times \vec{W}$ = signed area of parallelogram

area is positive if $\vec{V}$ is on the right of $\vec{W}$, forming a **counterclockwise** turn

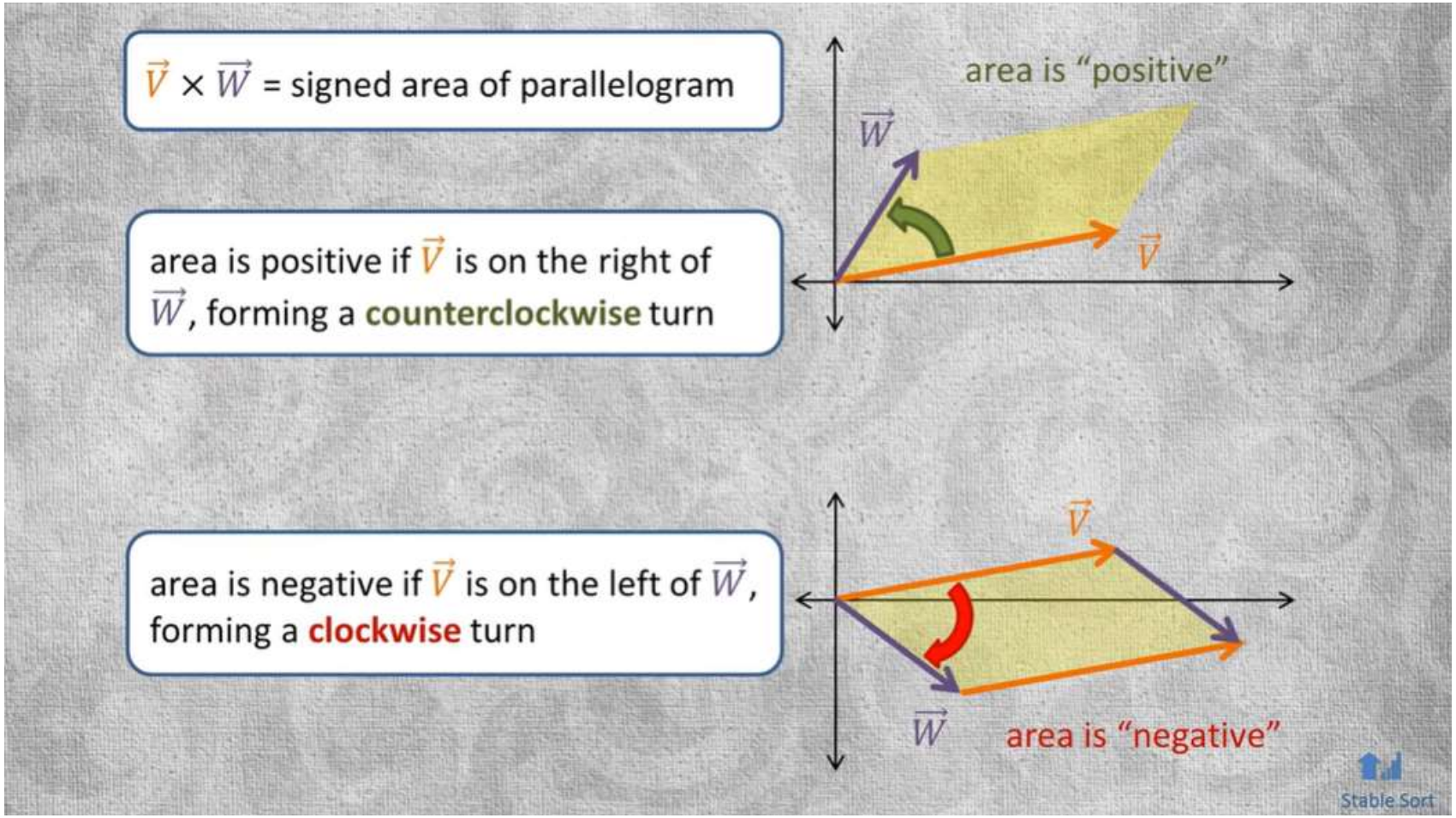


area is negative if $\vec{V}$ is on the left of $\vec{W}$, forming a **clockwise** turn



Implementation Details

- The cross product is perpendicular vector whose length = area of parallelogram
- While the sign tells the direction in which it is pointing
- But for our purposes, we just want to



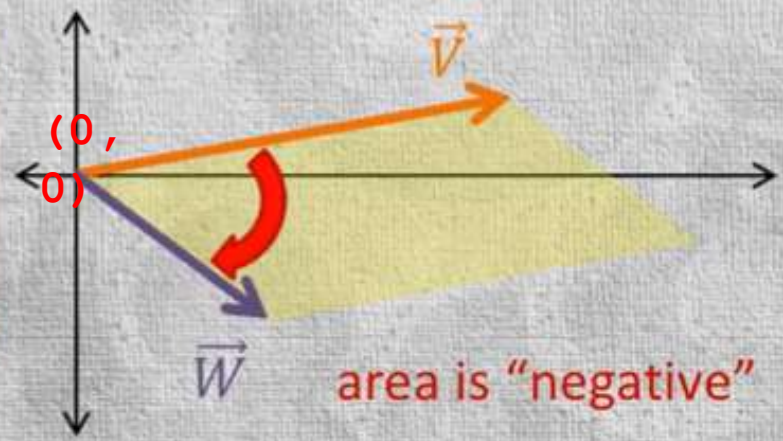
$$\vec{V} \times \vec{W} = \begin{bmatrix} V_x & W_x \\ V_y & W_y \end{bmatrix} = (V_x W_y) - (W_x V_y)$$

Example: $\vec{V}(5,2)$, $\vec{W}(3,4)$

$$= \begin{bmatrix} 5 & 3 \\ 2 & 4 \end{bmatrix} = (5 * 4) - (3 * 2) = 14$$

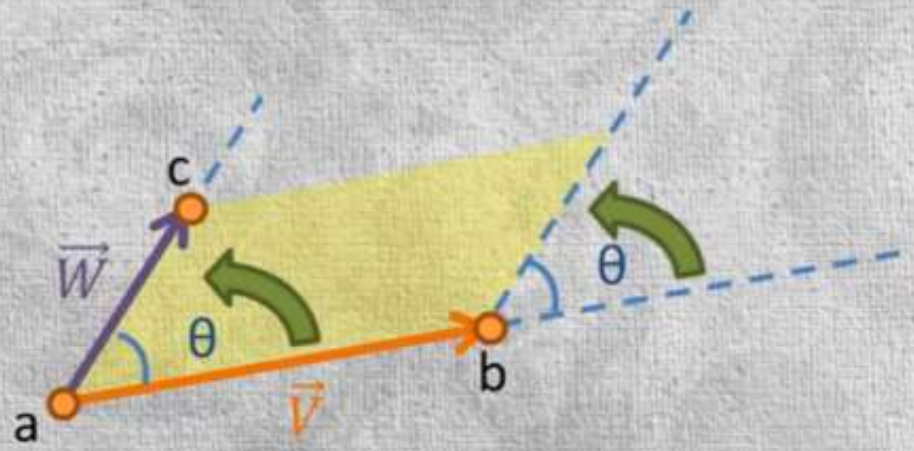
Example: $\vec{V}(5,2)$, $\vec{W}(2,-2)$

$$= \begin{bmatrix} 5 & 2 \\ 2 & -2 \end{bmatrix} = (5 * -2) - (2 * 2) = -14$$



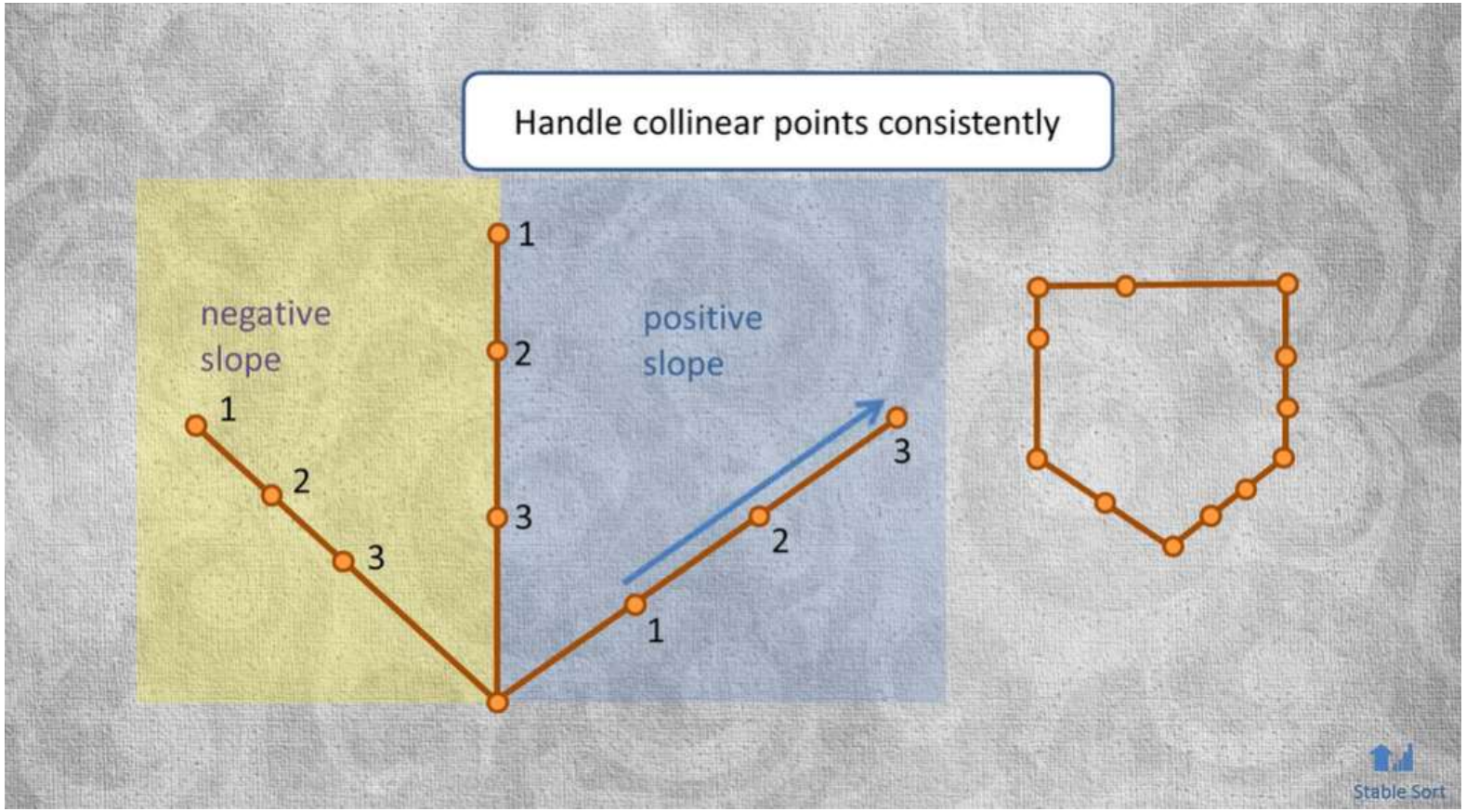
Stable Sort

```
int ccw(Point a, Point b, Point c) {
    float area = (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);

    if (area < 0) return -1; // clockwise

    if (area > 0) return 1; // counter-clockwise

    return 0; // collinear
}
```

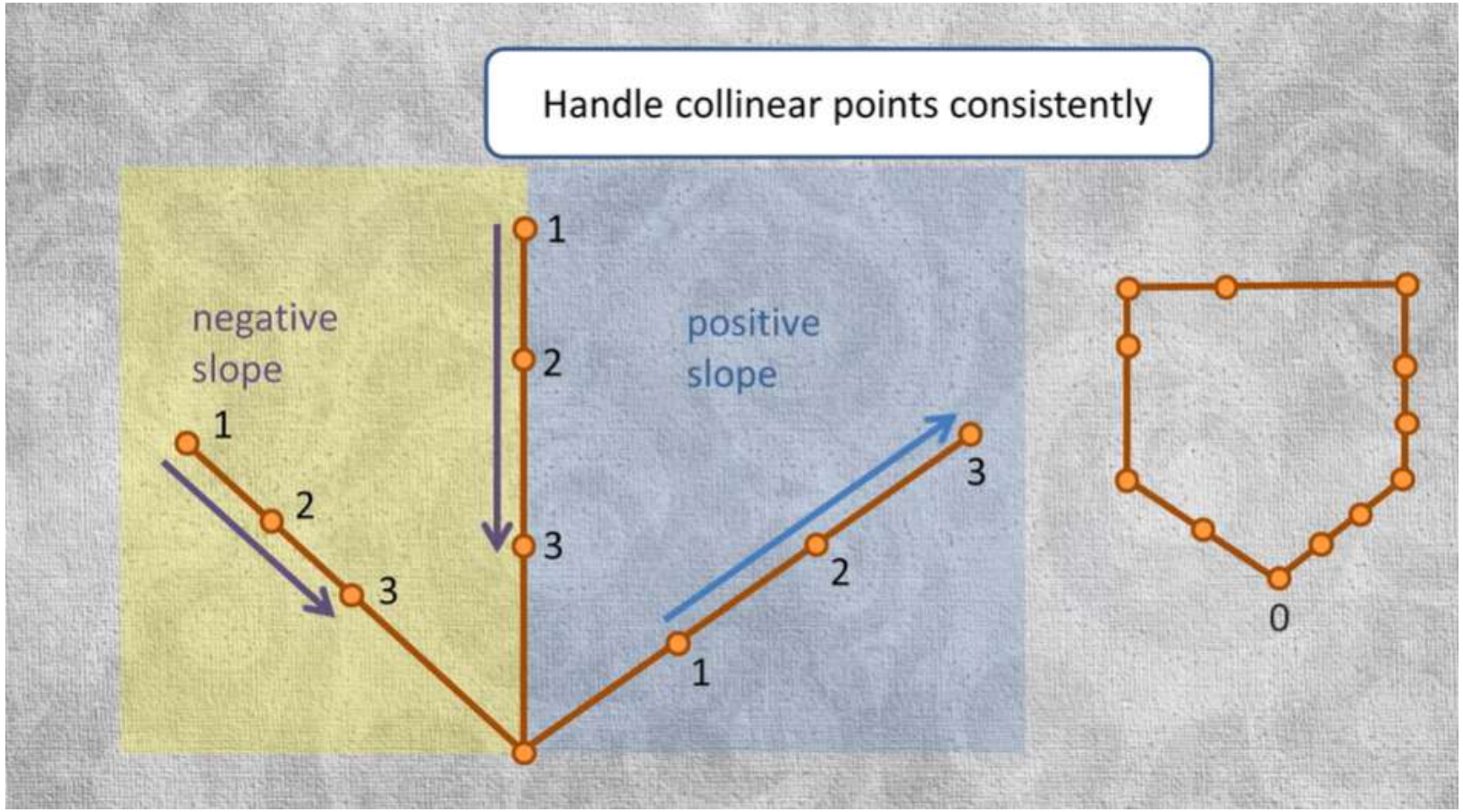


This formula can also be used to sort the points. The only thing needs to be cautious of is handling points that are collinear. Since we want to keep only peripheral points.

Make sure that the comparator orders the points from Left to Right when the slope of a line is positive



Make sure that the comparator orders the points from Right to Left when the slope of a line is vertical or negative

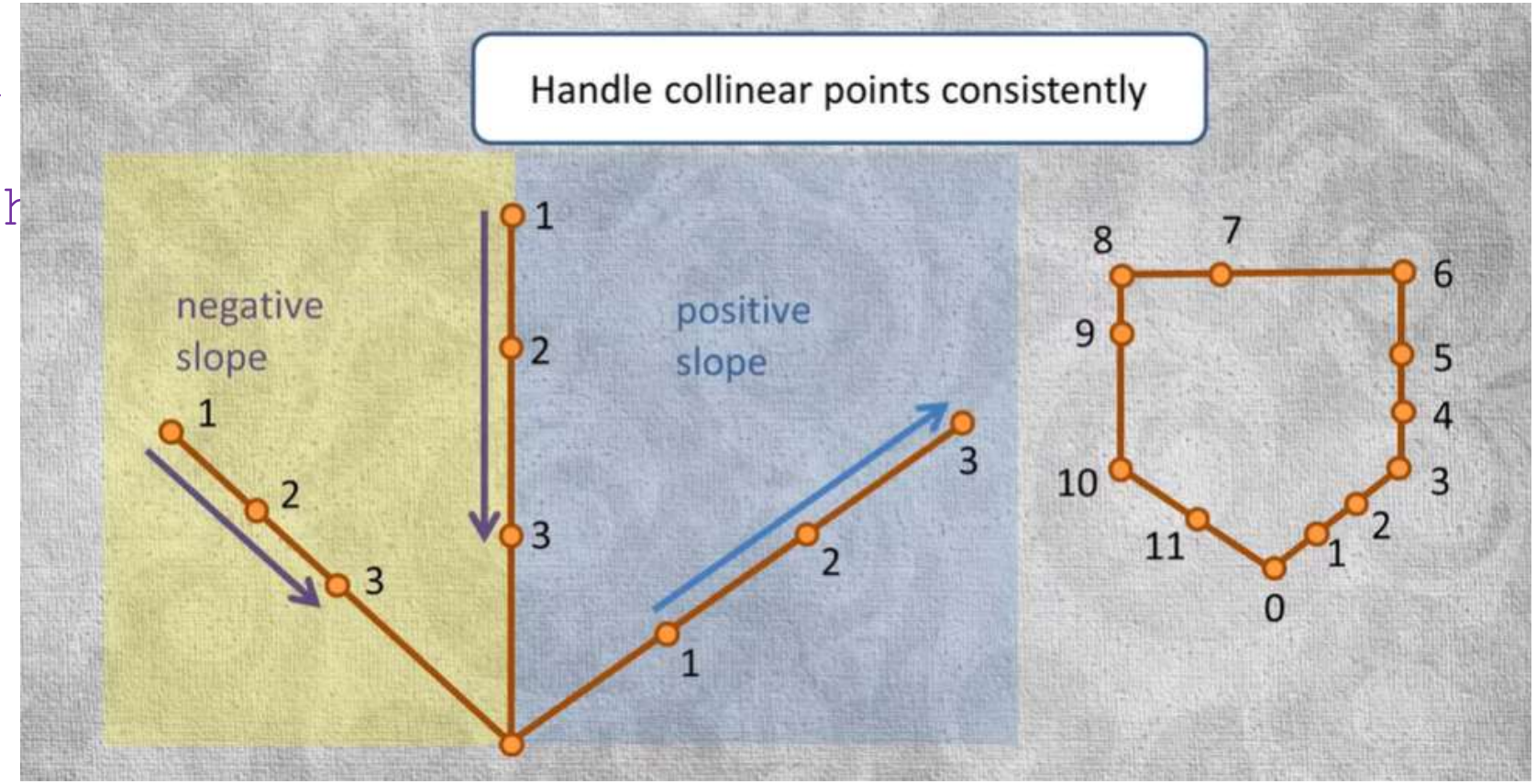


This is little tricky

esp. when dealing with

floating point

coordinates



```java
public List<Point> scan(List<? extends Point> points) {
    Deque<Point> stack = new ArrayDeque<Point>();
    Point minYPoint = GraphUtils.getMinY(points); // bottom most, left most point
    sortByAngle(points, minYPoint); // sort by angle with respect to minYPoint
    stack.push(points.get(0)); // 1st point is minYPoint, guaranteed to be on the hull
    stack.push(points.get(1)); // don't know about this one yet

    for (int i = 2, size = points.size(); i < size; i++) {
        Point next = points.get(i);
        Point p = stack.pop();
        while (stack.peek() != null && GraphUtils.ccw(stack.peek(), p, next) <= 0) {
            p = stack.pop(); // delete points that create clockwise turn
        }
        stack.push(p);
        stack.push(points.get(i));
    }
    Point p = stack.pop(); // last point pushed in could have been collinear
    if (GraphUtils.ccw(stack.peek(), p, minYPoint) > 0) {
        stack.push(p); // put it back, everything is fine
    }
    return new ArrayList<>(stack);
}
```

```java
public List<Point> scan(List<? extends Point> points) {
    Deque<Point> stack = new ArrayDeque<Point>();
    Point minYPoint = GraphUtils.getMinY(points); // bottom most, left most point
    sortByAngle(points, minYPoint); // sort by angle with respect to minYPoint
    stack.push(points.get(0)); // 1st point is minYPoint, guaranteed to be on the hull
    stack.push(points.get(1)); // don't know about this one yet

    for (int i = 2, size = points.size(); i < size; i++) {
        Point next = points.get(i);
        Point p = stack.pop();
        while (stack.peek() != null && GraphUtils.ccw(stack.peek(), p, next) <= 0) {
            p = stack.pop(); // delete points that create clockwise turn
        }
        stack.push(p);
        stack.push(points.get(i));
    }
    Point p = stack.pop(); // last point pushed in could have been collinear
    if (GraphUtils.ccw(stack.peek(), p, minYPoint) > 0) {
        stack.push(p); // put it back, everything is fine
    }
    return new ArrayList<>(stack);
}
```

O(n)

O(n log n)

O(n)

Jarvis March Algorithm

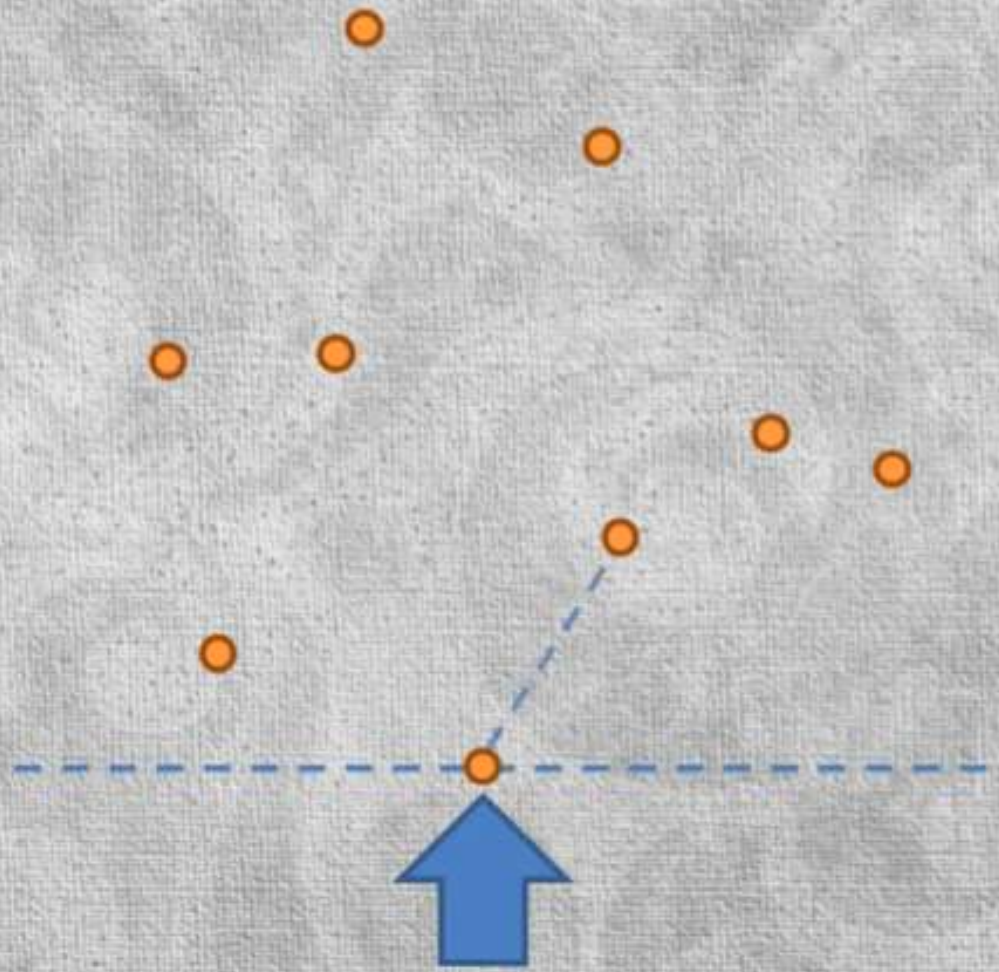
Jarvis march algorithm

select the point with the lowest Y coordinate - the 1st vertex

Instead of doing any kind of sorting, we just look through all the points again in the BRUTE FORCE way, to find the smallest counter clockwise angle in reference to the previous vertex.

Stable Sort

Jarvis march algorithm



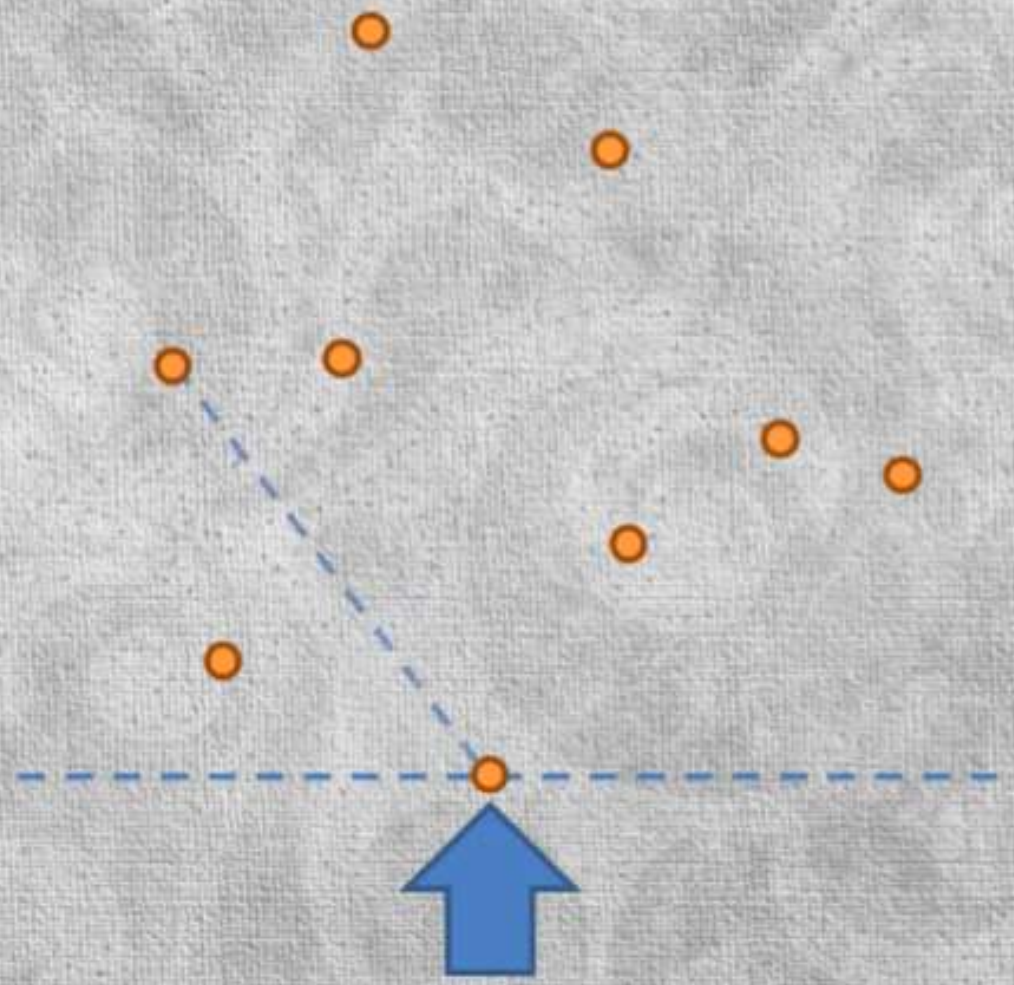
select the point with the lowest Y coordinate - the 1st vertex

select the point with smallest counterclockwise in reference to previous vertex

Instead of doing any kind of sorting, we just look through all the points again in the BRUTE FORCE way, to find the smallest counter clockwise angle in reference to the previous vertex.

Stable Sort

Jarvis march algorithm



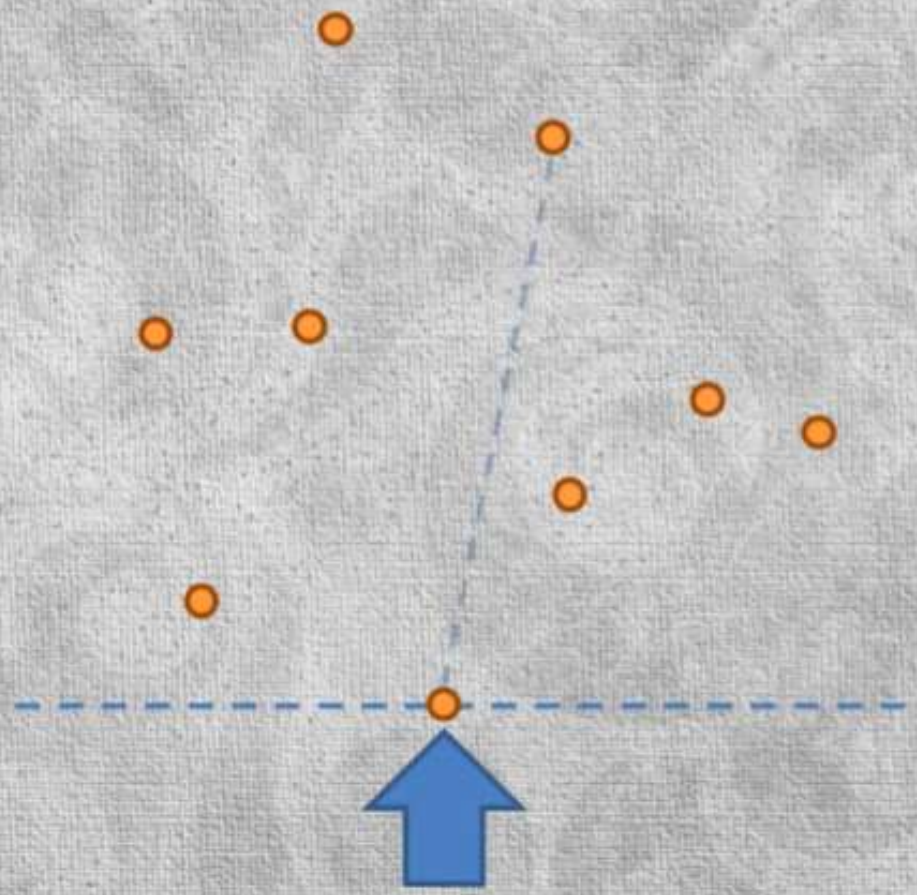
select the point with the lowest Y coordinate - the 1st vertex

select the point with smallest counterclockwise in reference to previous vertex

Instead of doing any kind of sorting, we just look through all the points again in the BRUTE FORCE way, to find the smallest counter clockwise angle in reference to the previous vertex.

Stable Sort

Jarvis march algorithm



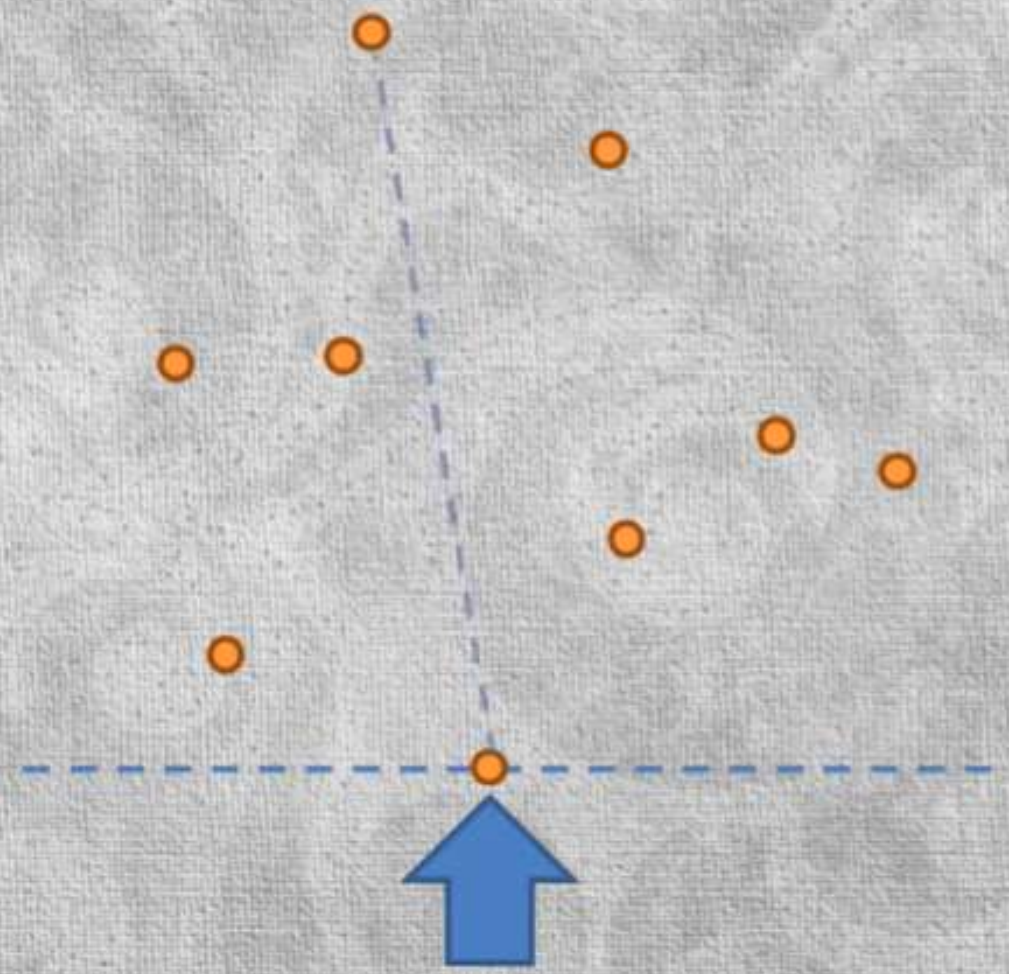
select the point with the lowest Y coordinate - the 1st vertex

select the point with smallest counterclockwise in reference to previous vertex

Instead of doing any kind of sorting, we just look through all the points again in the BRUTE FORCE way, to find the smallest counter clockwise angle in reference to the previous vertex.

Stable Sort

Jarvis march algorithm



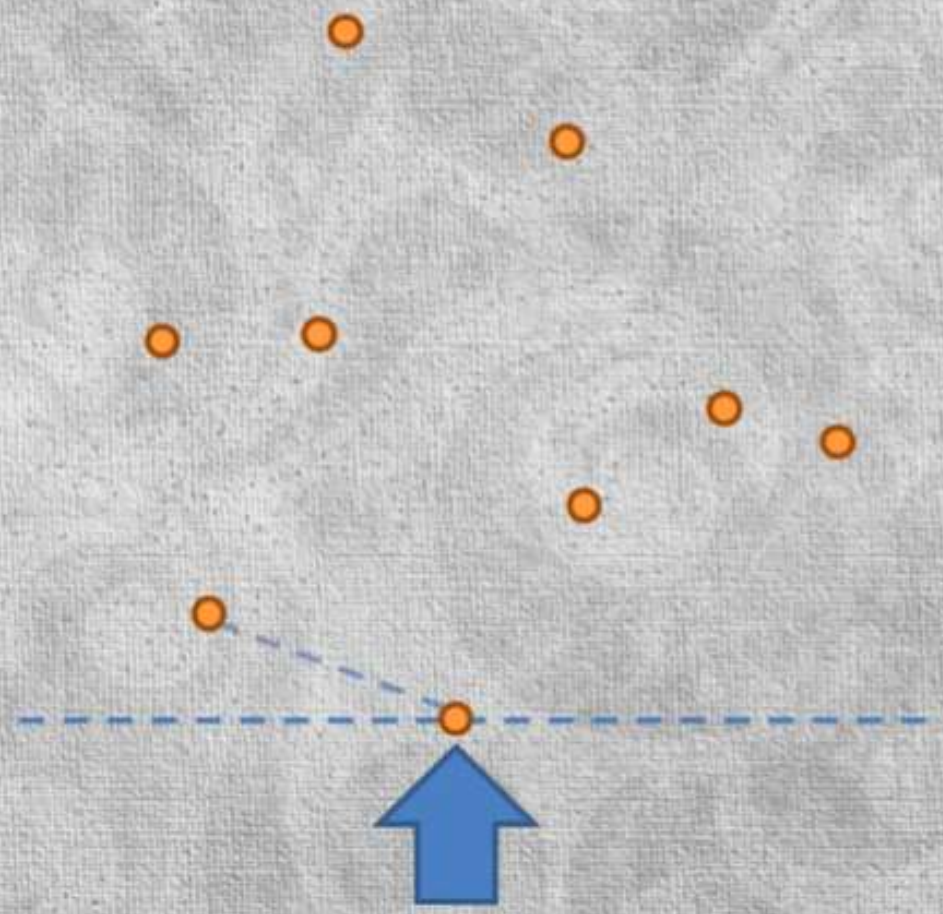
select the point with the lowest Y coordinate - the 1st vertex

select the point with smallest counterclockwise in reference to previous vertex

Instead of doing any kind of sorting, we just look through all the points again in the BRUTE FORCE way, to find the smallest counter clockwise angle in reference to the previous vertex.

Stable Sort

Jarvis march algorithm



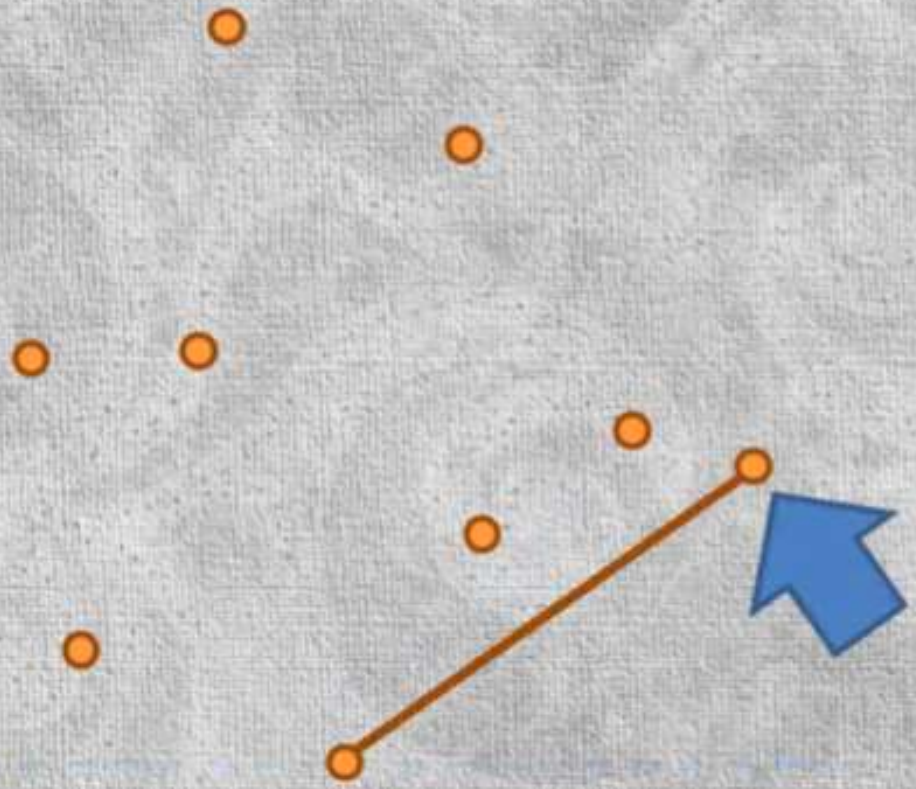
select the point with the lowest Y coordinate - the 1st vertex

select the point with smallest counterclockwise in reference to previous vertex

Instead of doing any kind of sorting, we just look through all the points again in the BRUTE FORCE way, to find the smallest counter clockwise angle in reference to the previous vertex.

Stable Sort

Jarvis march algorithm



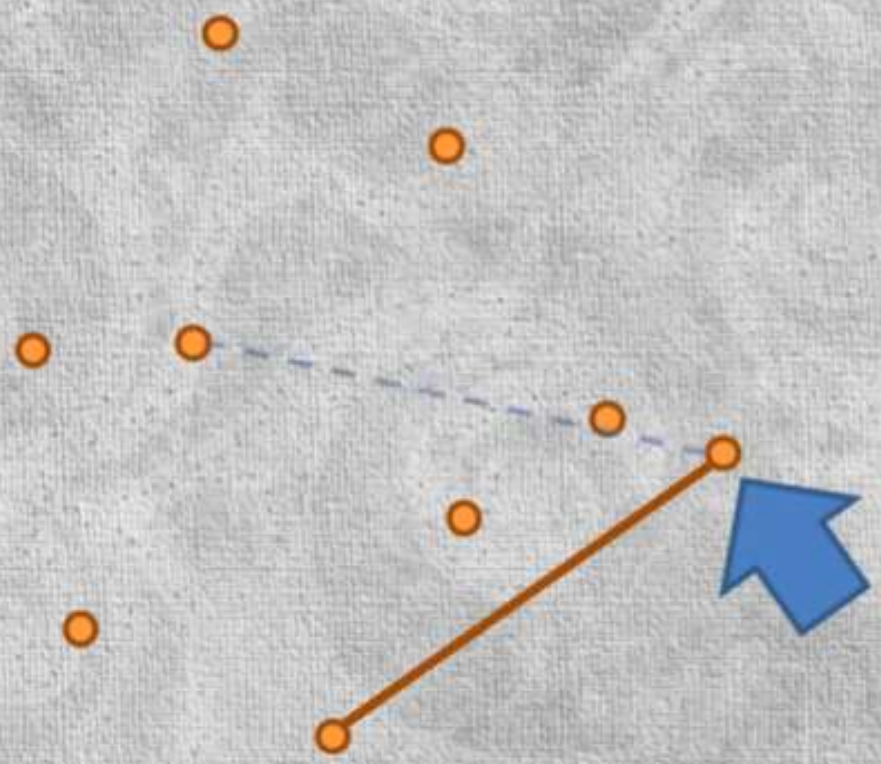
select the point with the lowest Y coordinate - the 1st vertex

select the point with smallest counterclockwise in reference to previous vertex

Instead of doing any kind of sorting, we just look through all the points again in the BRUTE FORCE way, to find the smallest **counter clockwise** angle in reference to the previous vertex.

Stable Sort

Jarvis march algorithm



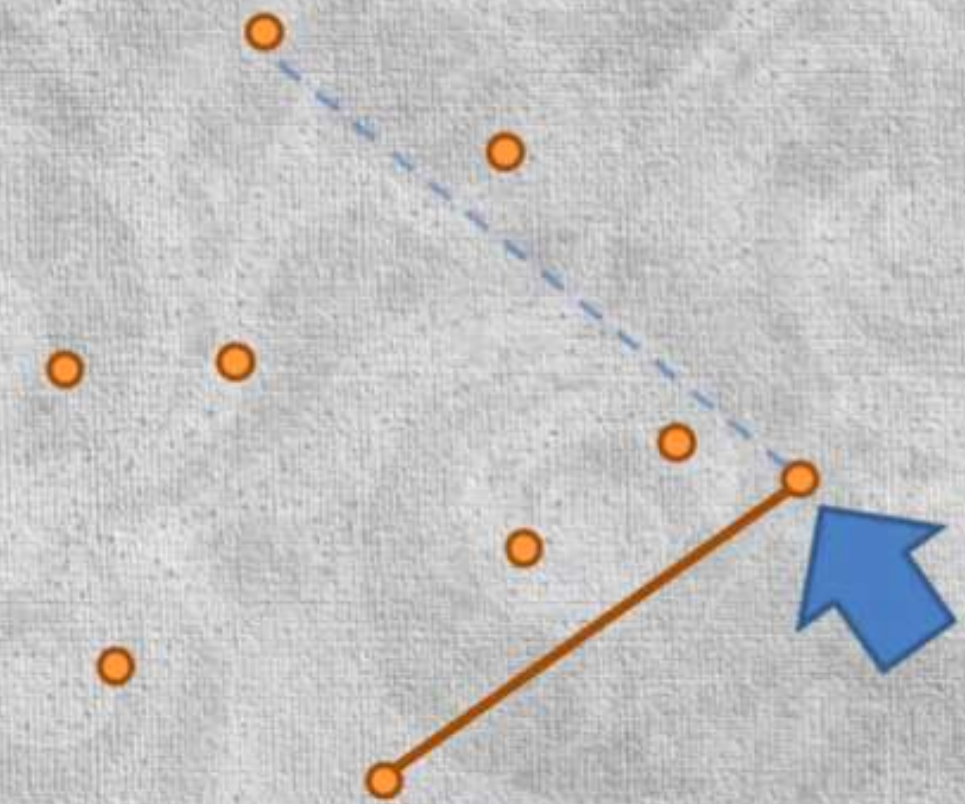
select the point with the lowest Y coordinate - the 1st vertex

select the point with smallest counterclockwise in reference to previous vertex

repeat until reaching the starting vertex

Repeat the process with new point

Jarvis march algorithm

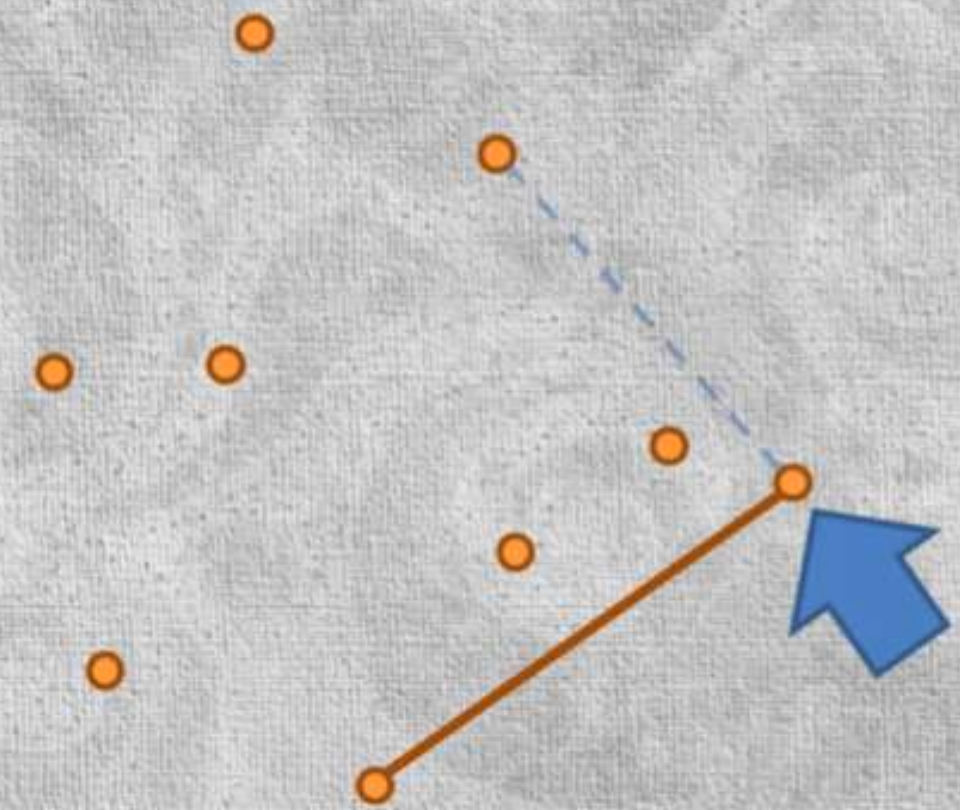


select the point with the lowest Y coordinate - the 1st vertex

select the point with smallest counterclockwise in reference to previous vertex

repeat until reaching the starting vertex

Jarvis march algorithm

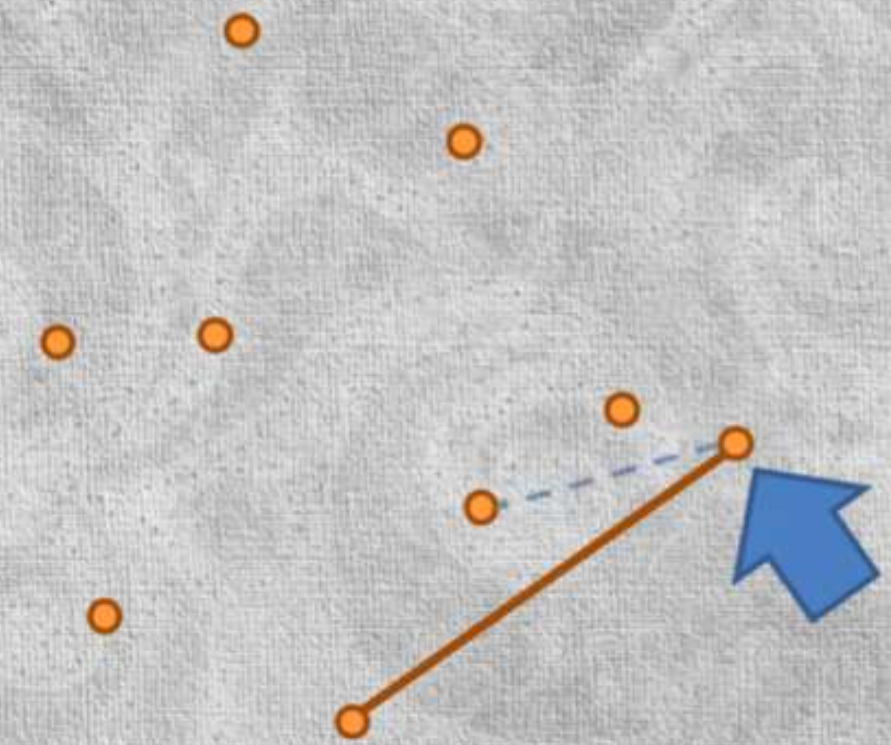


select the point with the lowest Y coordinate - the 1st vertex

select the point with smallest counterclockwise in reference to previous vertex

repeat until reaching the starting vertex

Jarvis march algorithm

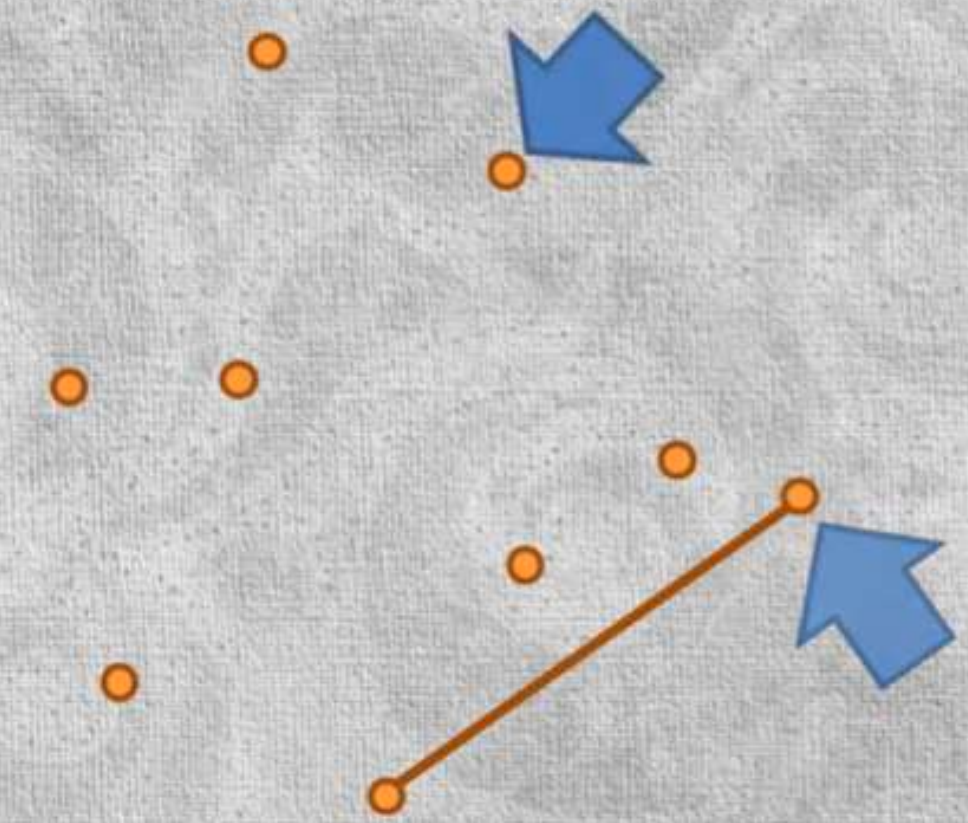


select the point with the lowest Y coordinate - the 1st vertex

select the point with smallest counterclockwise in reference to previous vertex

repeat until reaching the starting vertex

Jarvis march algorithm



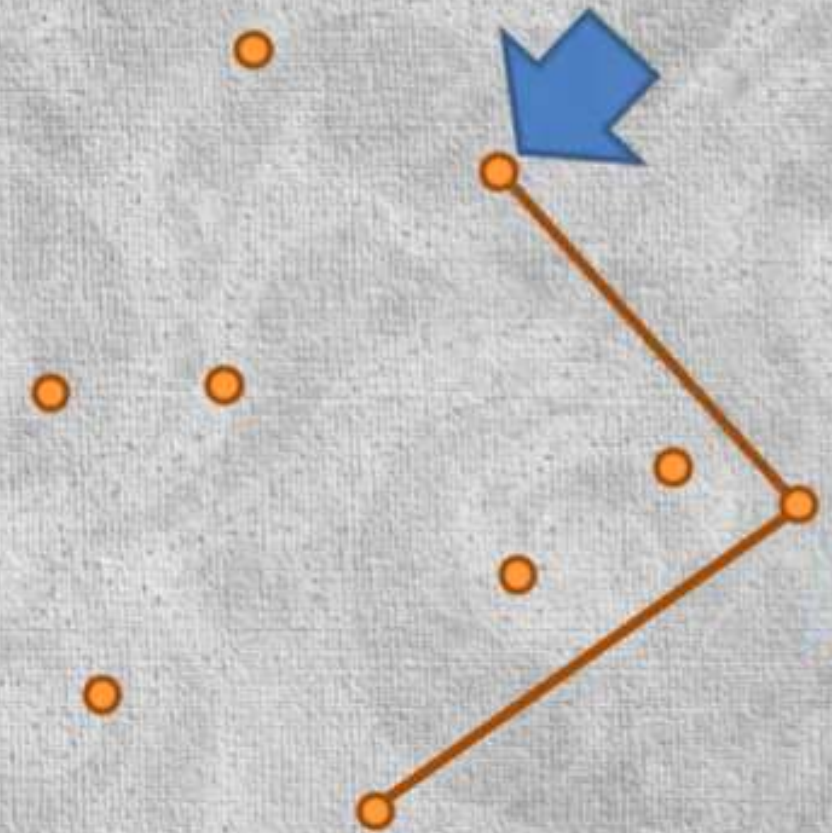
select the point with the lowest Y coordinate - the 1st vertex

select the point with smallest counterclockwise in reference to previous vertex

repeat until reaching the starting vertex

find the smallest **counter clockwise** angle in reference to the previous vertex.

Jarvis march algorithm

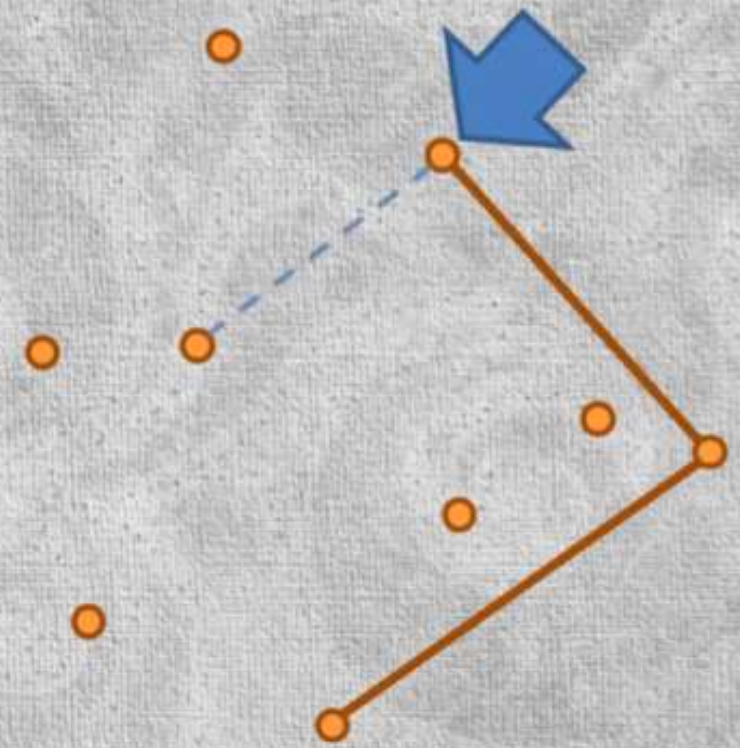


select the point with the lowest Y coordinate - the 1st vertex

select the point with smallest counterclockwise in reference to previous vertex

repeat until reaching the starting vertex

Jarvis march algorithm



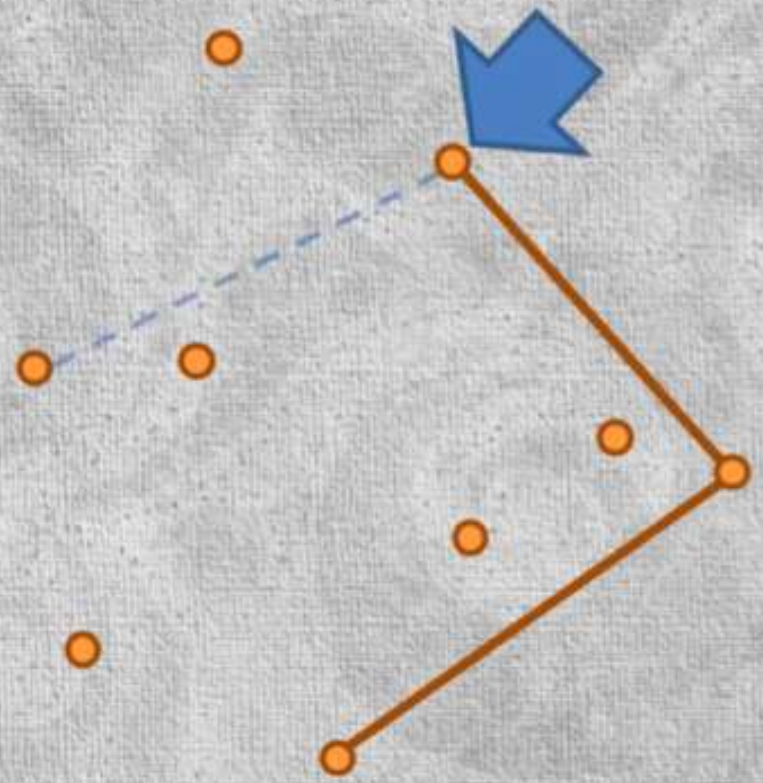
select the point with the lowest Y coordinate - the 1st vertex

select the point with smallest counterclockwise in reference to previous vertex

repeat until reaching the starting vertex

Repeat the process with new point

Jarvis march algorithm



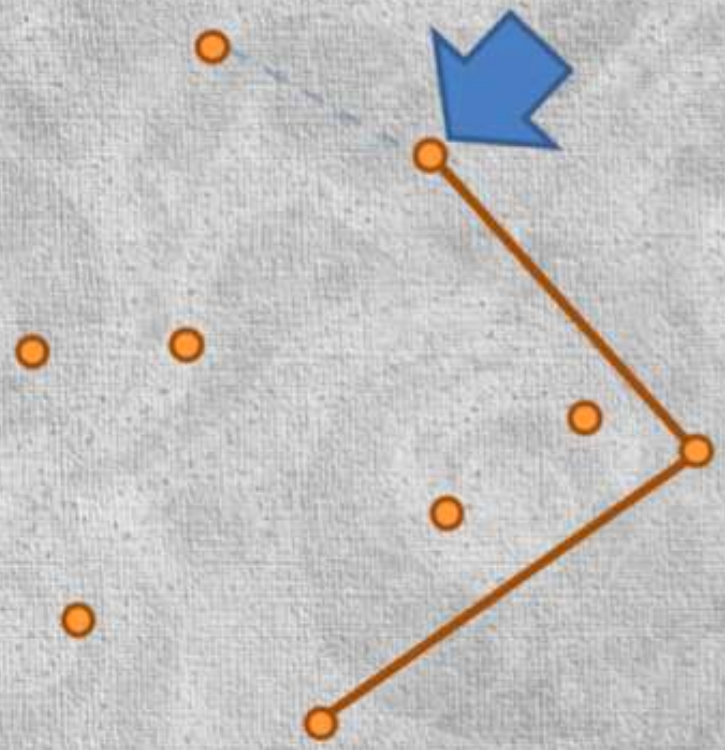
select the point with the lowest Y coordinate - the 1st vertex

select the point with smallest counterclockwise in reference to previous vertex

repeat until reaching the starting vertex

Repeat the process with new point

Jarvis march algorithm



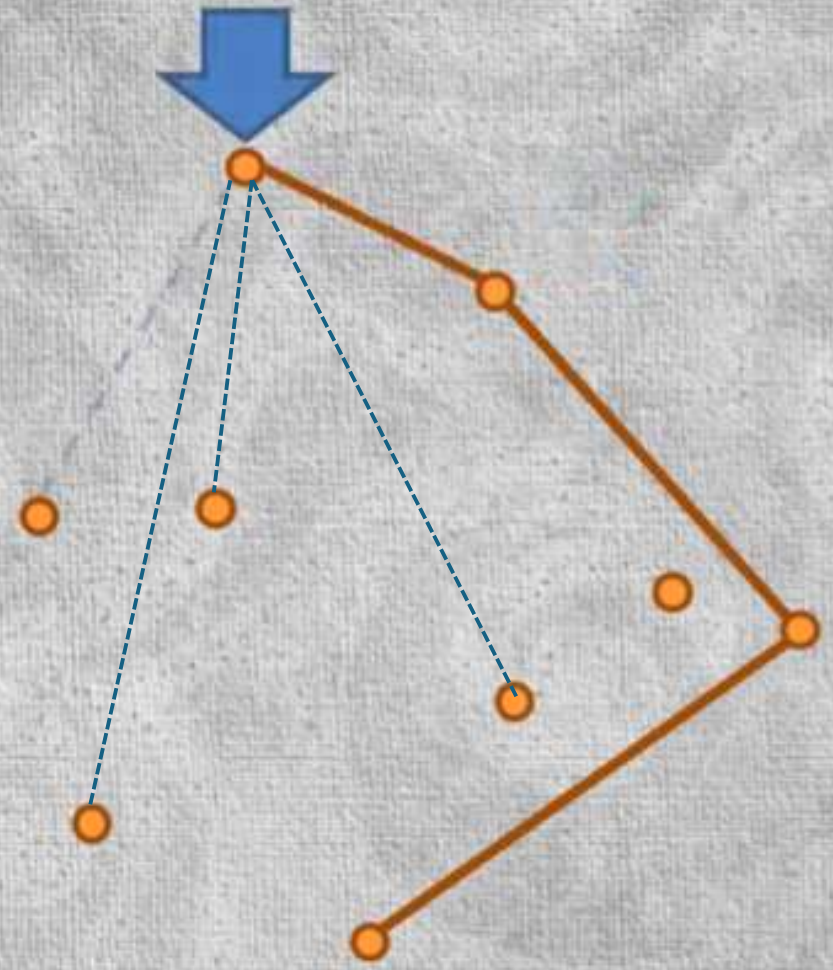
select the point with the lowest Y coordinate - the 1st vertex

select the point with smallest counterclockwise in reference to previous vertex

repeat until reaching the starting vertex

Repeat the process with new point

Jarvis march algorithm



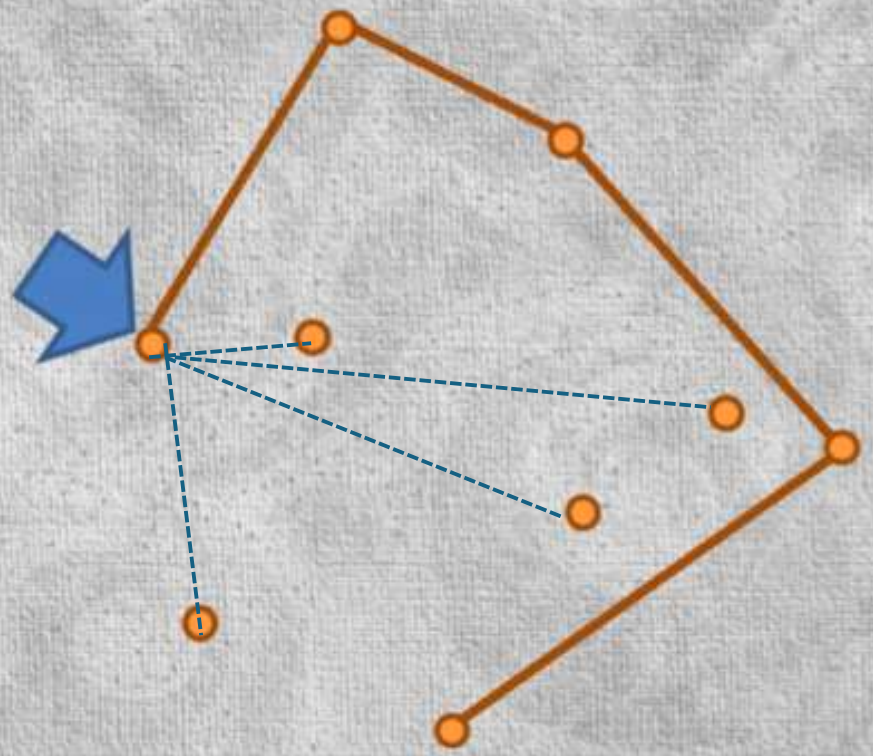
select the point with the lowest Y coordinate - the 1st vertex

select the point with smallest counterclockwise in reference to previous vertex

repeat until reaching the starting vertex

Repeat the process with new point

Jarvis march algorithm



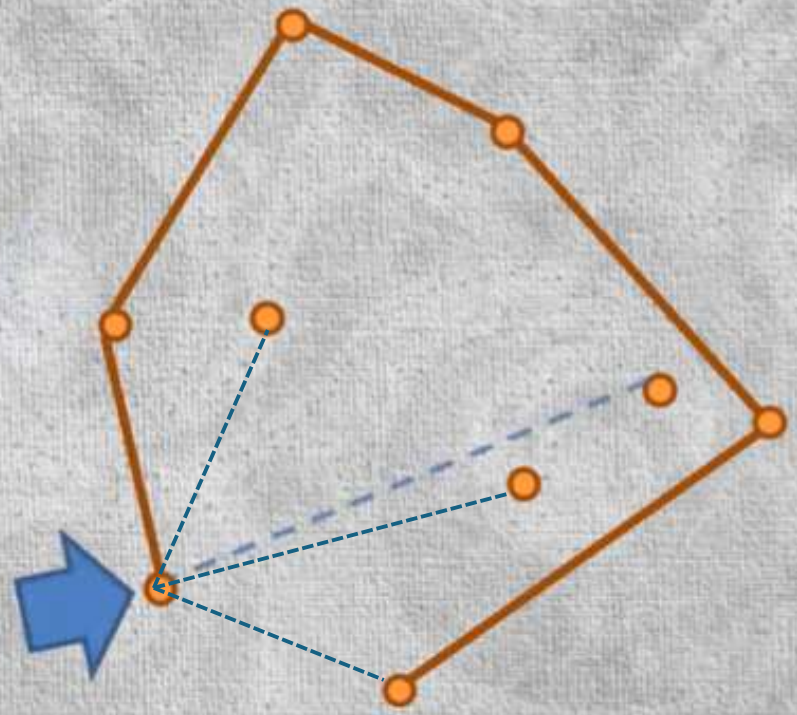
select the point with the lowest Y coordinate - the 1st vertex

select the point with smallest counterclockwise in reference to previous vertex

repeat until reaching the starting vertex

Repeat the process with new point

Jarvis march algorithm



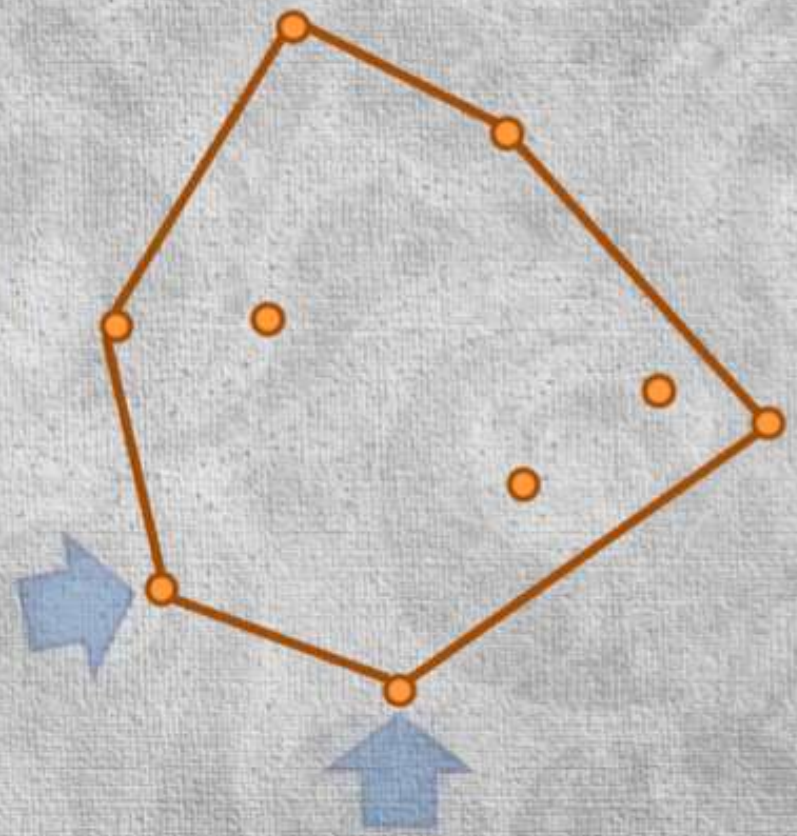
select the point with the lowest Y coordinate - the 1st vertex

select the point with smallest counterclockwise in reference to previous vertex

repeat until reaching the starting vertex

Repeat the process with new point

Jarvis march algorithm



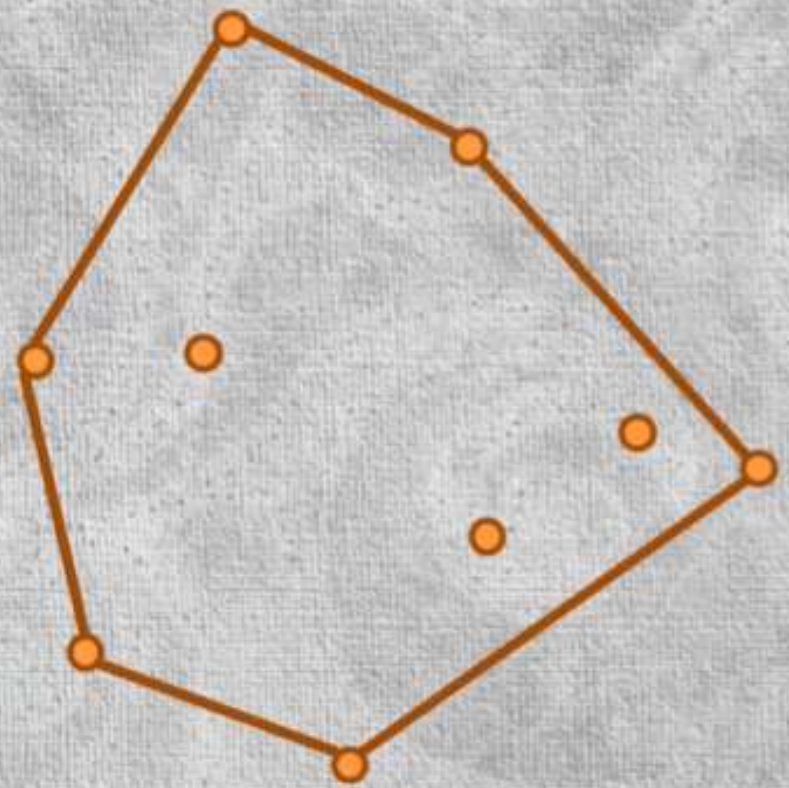
select the point with the lowest Y coordinate - the 1st vertex

select the point with smallest counterclockwise in reference to previous vertex

repeat until reaching the starting vertex

Repeat the process with new point through all the points until all the vertices are determined and it gets back to the starting point.

Jarvis march algorithm

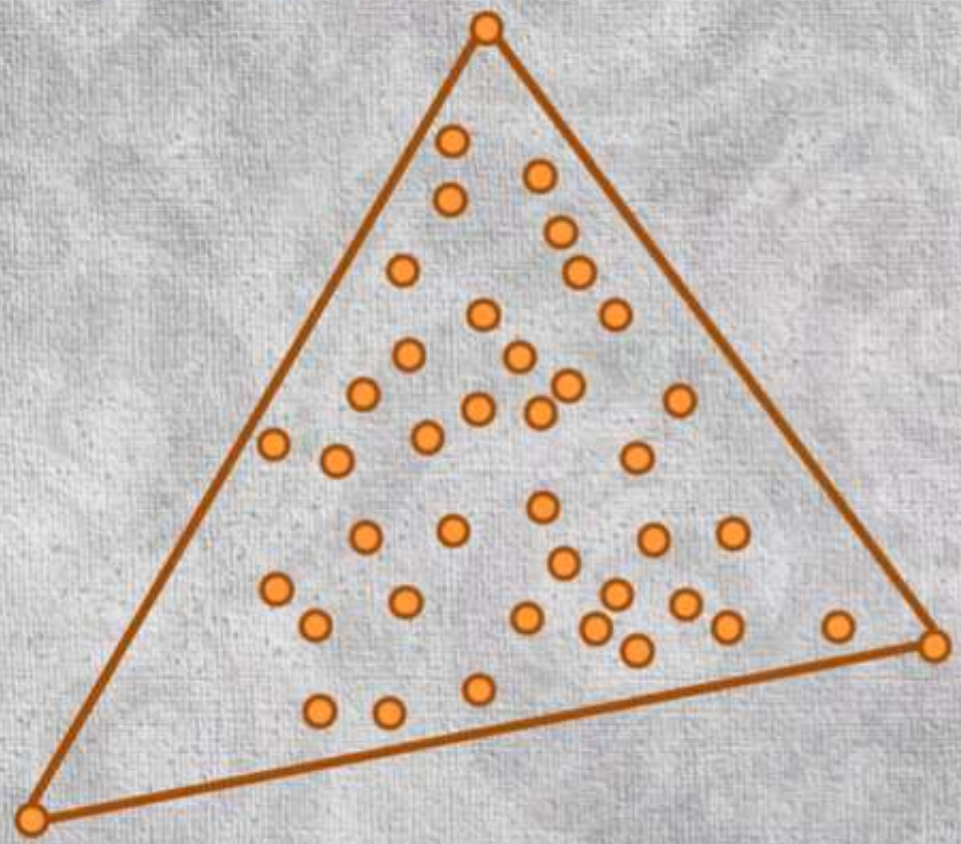


select the point with the lowest Y coordinate - the 1st vertex

select the point with smallest counterclockwise in reference to previous vertex

repeat until reaching the starting vertex

Jarvis march algorithm



O(n h) running time, where h is the number of vertices

```java
public List<Point> march(Collection<? extends Point> points) {
    List<Point> hull = new ArrayList<>();
    Point startingPoint = GraphUtils.getMinY(points); // bottom most, left most point is guaranteed to be on the hull
    hull.add(startingPoint);
    Point prevVertex = startingPoint;

    while (true) {
        Point candidate = null;
        for (Point point : points) {
            if (point == prevVertex) continue;
            if (candidate == null) {
                candidate = point;
                continue;
            }
            int ccw = GraphUtils.ccw(prevVertex, candidate, point);
            if (ccw == 0 && GraphUtils.dist(prevVertex, candidate) < GraphUtils.dist(prevVertex, point)) {
                candidate = point; // collinear tie is decided by the distance
            } else if (ccw < 0) {
                /*
                 * if made a clockwise turn, then found a better point that makes a smaller
                 * counterclockwise angle in reference to the previous vertex
                 */
                candidate = point;
            }
        }

        if (candidate == startingPoint) break; // came back to the starting point, so we are done
        hull.add(candidate);
        prevVertex = candidate;
    }
    return hull;
}
```

same function that makes use of the cross product

Function ccw: Counter Clockwise

CCW

- ccw: makes use of the cross product
- Little bonus: dealing with collinear points is also a little easier b'coz we need to pick the point that is furthest away distance wise without needing to worry about the slope of the line.
- Both these algorithms were invented in early 1970s.

In computational geometry, "H" typically refers to the number of points in the convex hull, which is a subset of the total number of points n in a given set. The notation $O(n \log H)$ represents the time complexity of Chan's algorithm for finding the convex hull, where n is the total number of input points, and H is the number of points that form the actual convex hull.